Intro2SE – Design – 4

PathToGrad

*The student team is required to complete the **Design Document** for the assigned course project, following the attached template.*

Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS

Table of Contents

Software Design

Objectives

This document focus on the following topics:

- ✔ Completing the Software Design Document with the following contents:
  - Conceptual Model
  - Architectural Design
  - Data Design
  - User Interface Design
- ✔ Understanding the Software Design Document.
- ✔ This document will be used as input for AI tools to verify the quality of subsequent project artifacts.

All project artifacts must remain consistent and synchronized.
For example, if the project proposal is modified during this phase, a new version of the proposal must be documented.
By the conclusion of the project, all versions of every artifact must be submitted to demonstrate the evolution of your work.

1 Member Contribution Assessment

24127427 - Trần Nhựt Đăng Khoa - 25%

- **Task Evidence:**

Task 1: Design Phase Planning and Task Distribution

Description: Defined the work structure for the Software Design phase and distributed responsibilities according to each member's project role. Retained sprint management, document integration, quality assurance, and the Reflective Report.

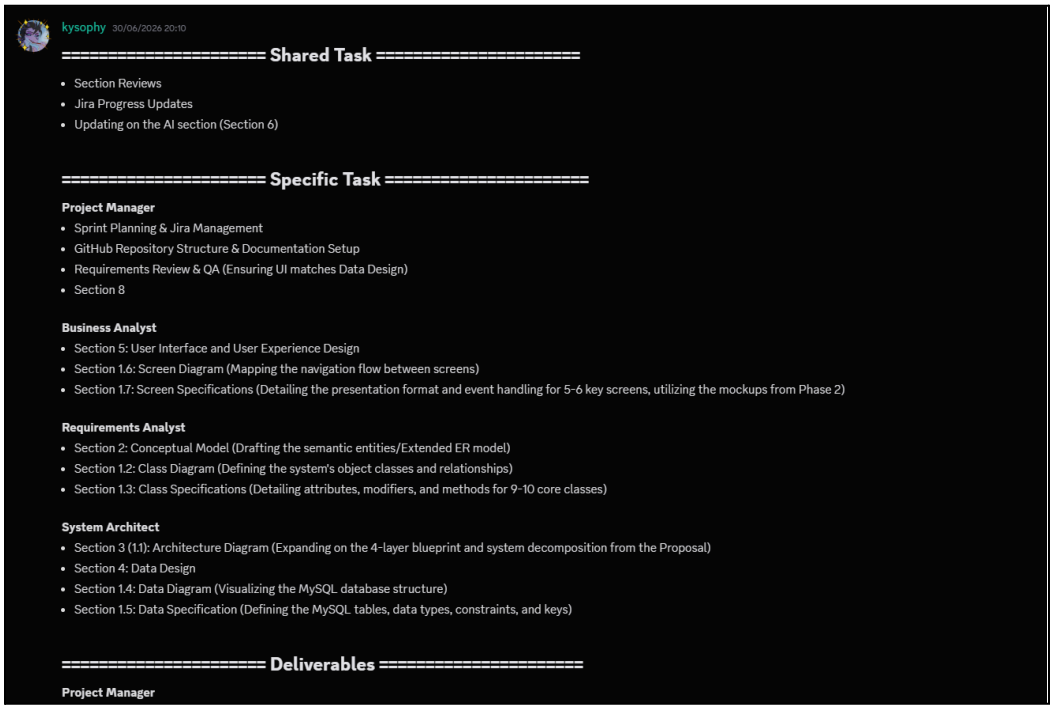
Task 2: Sprint Tracking and Team Coordination

Description: Organized and monitored the design-phase tasks through Jira and team communication channels. Followed the completion status of each assigned section, communicated revision requests, and coordinated the dependency order between the Conceptual Model, Class Diagram, Data Design, and Screen Design. Ensured that team members received clear instructions regarding the expected deliverables and submission format.

Task 3: Cross-Artifact Consistency Review and Quality Assurance

Description: Conducted QA across the completed design artifacts. Reviewed the Architecture Diagram, Class Specifications, Data Diagram, Data Specifications, Screen Diagram, and Screen Specifications. Verified that the user interface elements and actions could be supported by the database design. Coordinated corrections involving User and Student Profile separation, Academic Record and Course Attempt relationships, Course Offering, Class Section and Section Meeting scheduling.

- **Visual Evidence:**



kysophy 30/06/2026 20:10

========== Shared Task ==========

- Section Reviews
- Jira Progress Updates
- Updating on the AI section (Section 6)

========== Specific Task ==========

Project Manager

- Sprint Planning & Jira Management
- GitHub Repository Structure & Documentation Setup
- Requirements Review & QA (Ensuring UI matches Data Design)
- Section 8

Business Analyst

- Section 5: User Interface and User Experience Design
- Section 1.6: Screen Diagram (Mapping the navigation flow between screens)
- Section 1.7: Screen Specifications (Detailing the presentation format and event handling for 5-6 key screens, utilizing the mockups from Phase 2)

Requirements Analyst

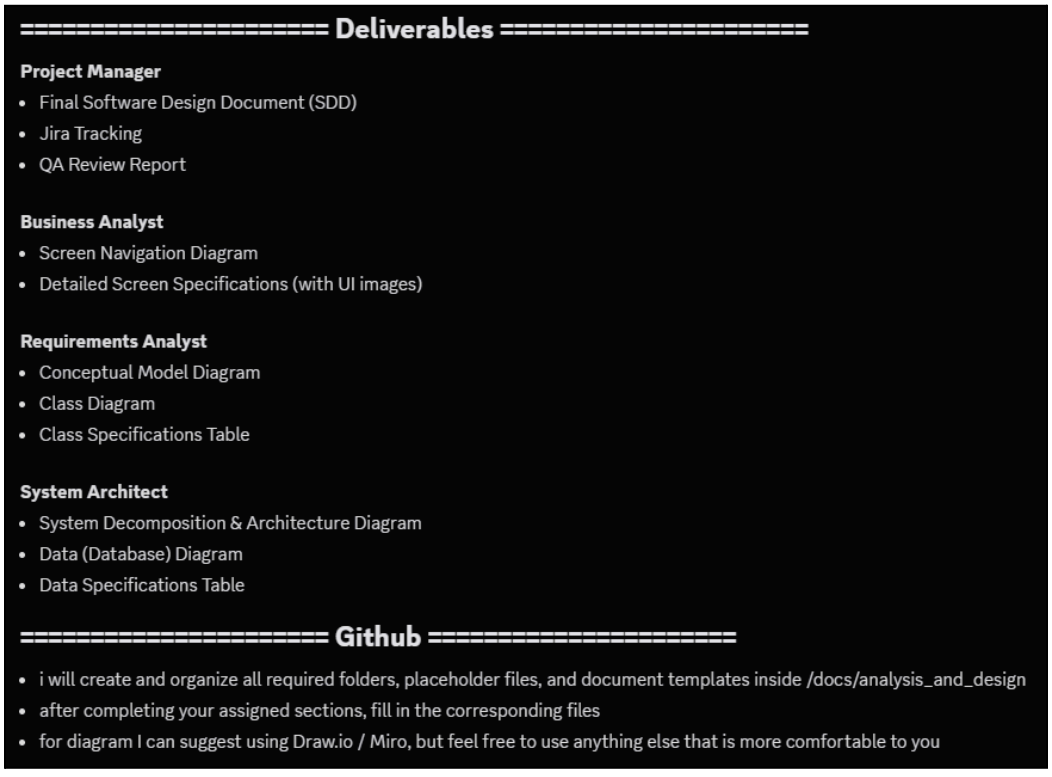
- Section 2: Conceptual Model (Drafting the semantic entities/Extended ER model)
- Section 1.2: Class Diagram (Defining the system's object classes and relationships)
- Section 1.3: Class Specifications (Detailing attributes, modifiers, and methods for 9-10 core classes)

System Architect

- Section 3 (1.1): Architecture Diagram (Expanding on the 4-layer blueprint and system decomposition from the Proposal)
- Section 4: Data Design
- Section 1.4: Data Diagram (Visualizing the MySQL database structure)
- Section 1.5: Data Specification (Defining the MySQL tables, data types, constraints, and keys)

========== Deliverables ==========

Project Manager



========== Deliverables ==========

Project Manager

- Final Software Design Document (SDD)
- Jira Tracking
- QA Review Report

Business Analyst

- Screen Navigation Diagram
- Detailed Screen Specifications (with UI images)

Requirements Analyst

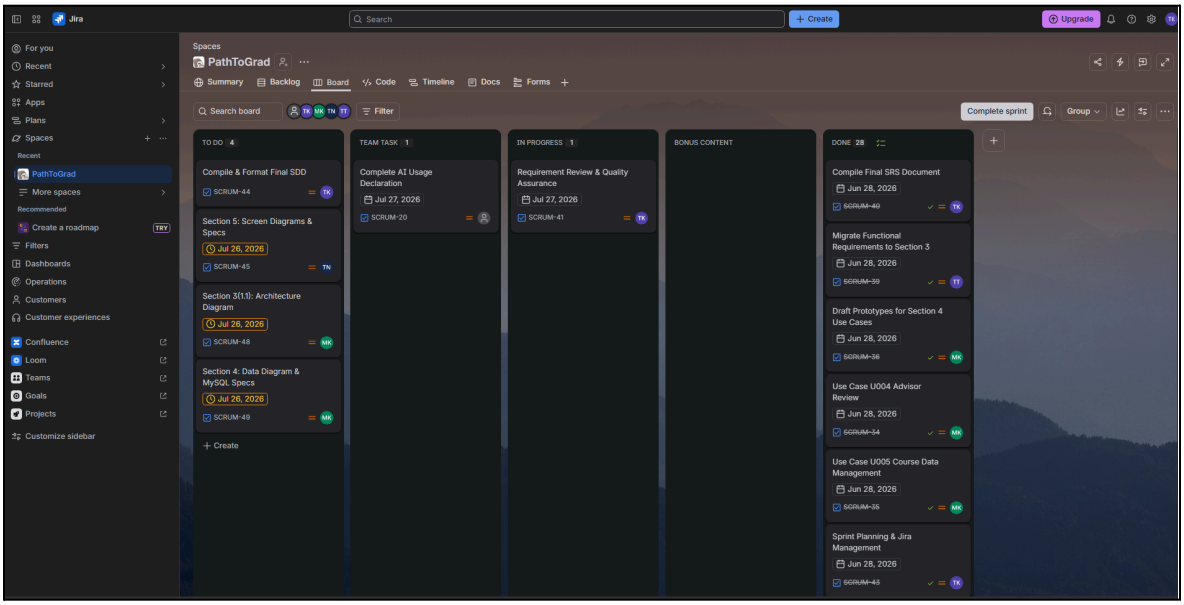
- Conceptual Model Diagram
- Class Diagram
- Class Specifications Table

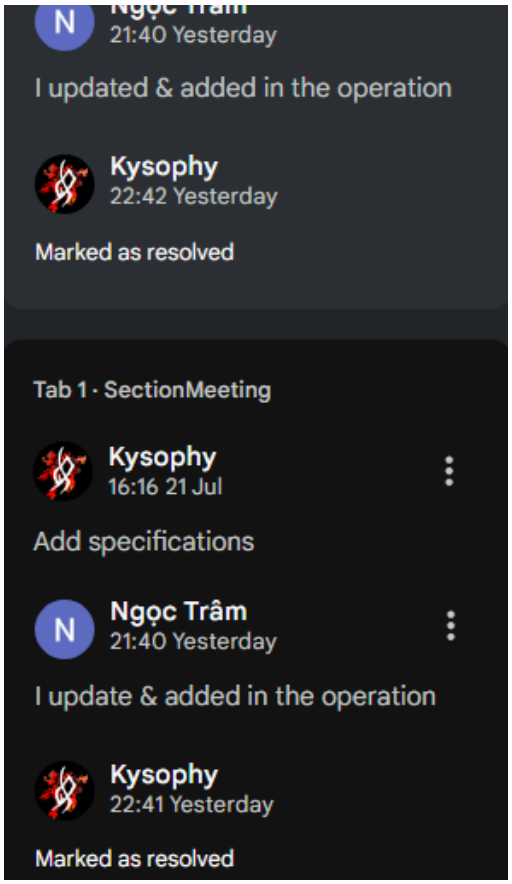
System Architect

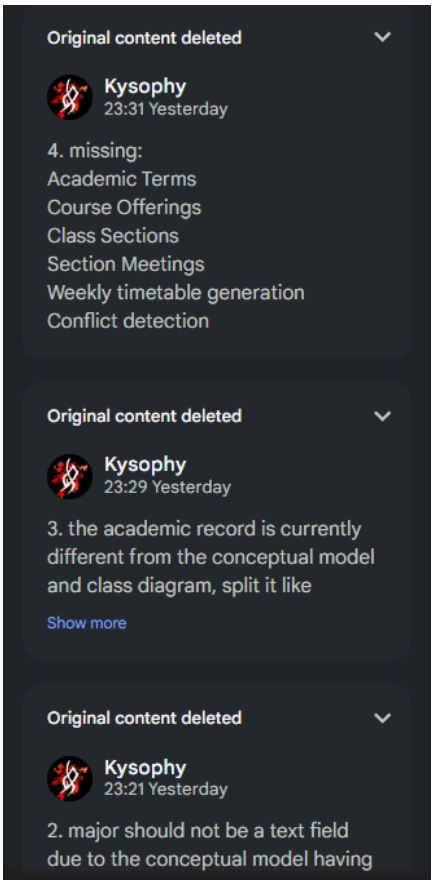
- System Decomposition & Architecture Diagram
- Data (Database) Diagram
- Data Specifications Table

========== Github ==========

- i will create and organize all required folders, placeholder files, and document templates inside /docs/analysis_and_design
- after completing your assigned sections, fill in the corresponding files
- for diagram I can suggest using Draw.io / Miro, but feel free to use anything else that is more comfortable to you


Jira
Search
Create
Upgrade
For you
Recent
Starred
Apps
Plans
Spaces
PathToGrad
More spaces
Create a roadmap
Filters
Dashboards
Operations
Customers
Customer experiences
Confluence
Loom
Teams
Goals
Projects
Customize sidebar
Spaces
PathToGrad
Summary
Backlog
Board
Code
Timeline
Docs
Forms
Search board
Filter
Complete sprint
Group
TO DO 4
Compile & Format Final SDD
SCRUM-44
Section 5: Screen Diagrams & Specs
Jul 26, 2026
SCRUM-45
Section 3(11): Architecture Diagram
Jul 26, 2026
SCRUM-48
Section 4: Data Diagram & MySQL Specs
Jul 26, 2026
SCRUM-49
Create
TEAM TASK 1
Complete AI Usage Declaration
Jul 27, 2026
SCRUM-20
IN PROGRESS 1
Requirement Review & Quality Assurance
Jul 27, 2026
SCRUM-41
BONUS CONTENT
DONE 28
Compile Final SRS Document
Jun 28, 2026
Migrate Functional Requirements to Section 3
Jun 28, 2026
Draft Prototypes for Section 4 Use Cases
Jun 28, 2026
Use Case U004 Advisor Review
Jun 28, 2026
Use Case U005 Course Data Management
Jun 28, 2026
Sprint Planning & Jira Management
Jun 28, 2026


21:40 Yesterday
I updated & added in the operation
Kysophy
22:42 Yesterday
Marked as resolved
Tab 1 · SectionMeeting
Kysophy
16:16 21 Jul
Add specifications
Ngọc Trâm
21:40 Yesterday
I update & added in the operation
Kysophy
22:41 Yesterday
Marked as resolved


Original content deleted
Kysophy
23:31 Yesterday
4. missing:
Academic Terms
Course Offerings
Class Sections
Section Meetings
Weekly timetable generation
Conflict detection
Original content deleted
Kysophy
23:29 Yesterday
3. the academic record is currently different from the conceptual model and class diagram, split it like
Show more
Original content deleted
Kysophy
23:21 Yesterday
2. major should not be a text field due to the conceptual model having

24127543 - Nguyễn Trương Ngọc Thảo - 25%

- **Task Evidence:**

Task 1: Create Screen Diagram

Description: Specify the screen flow for each role of the user and draw diagram of screen transitions between them.

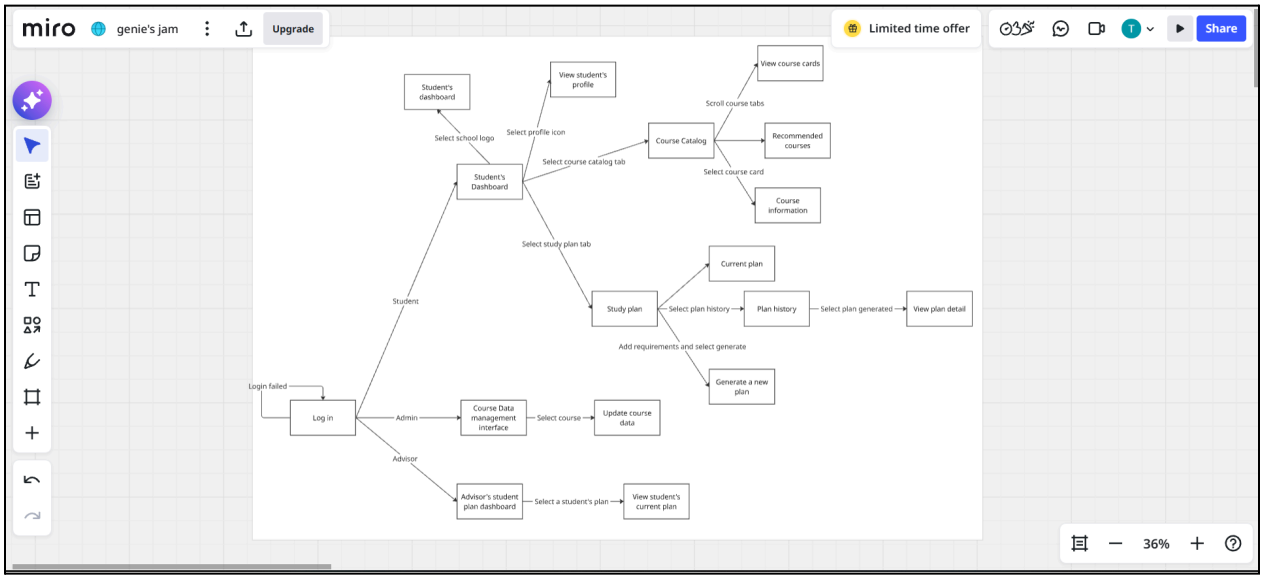
Task 2: Design demo User Interface

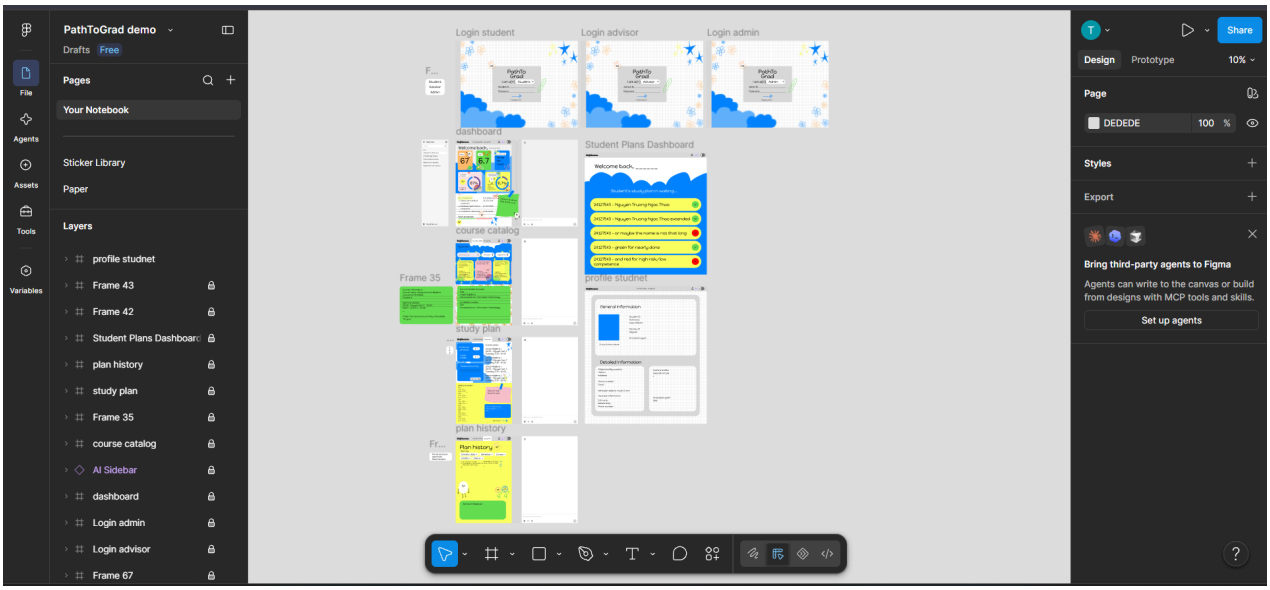
Description: Use Figma to design and mark interaction protocols for User Interface and User Experience.

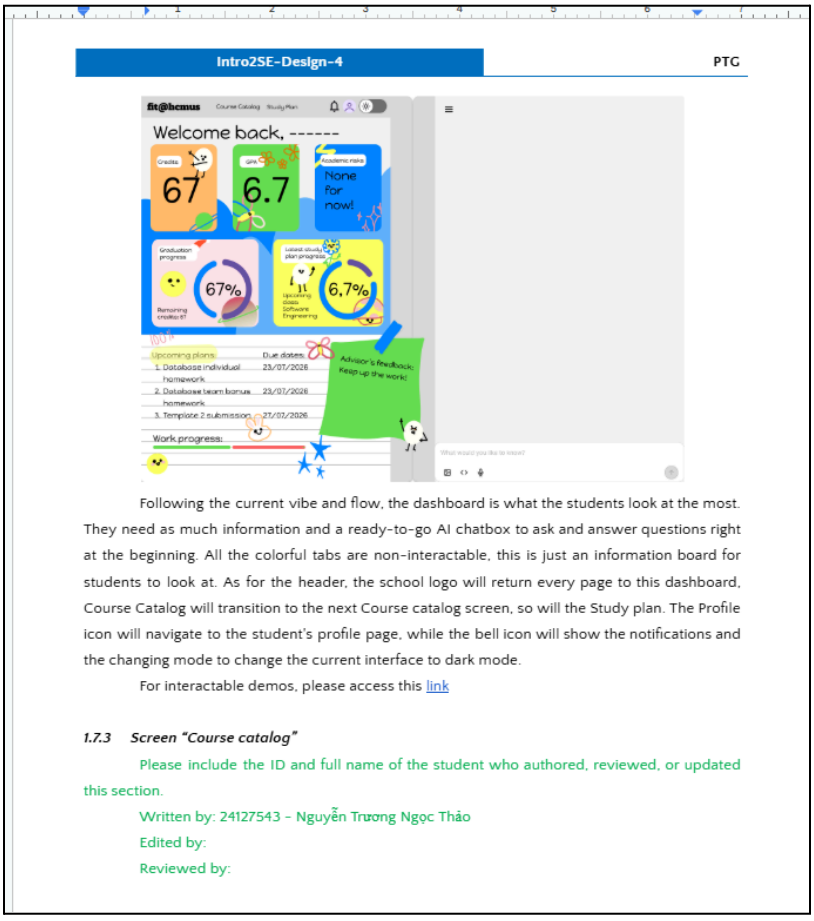
Task 3: Write Screen specification

Description: Specify the design idea, purpose of each interactive part of the screen.

- **Visual Evidence:**







Intro2SE-Design-4 PTG

Following the current vibe and flow, the dashboard is what the students look at the most. They need as much information and a ready-to-go AI chatbox to ask and answer questions right at the beginning. All the colorful tabs are non-interactable, this is just an information board for students to look at. As for the header, the school logo will return every page to this dashboard, Course Catalog will transition to the next Course catalog screen, so will the Study plan. The Profile icon will navigate to the student's profile page, while the bell icon will show the notifications and the changing mode to change the current interface to dark mode.

For interactable demos, please access this link

1.7.3 Screen "Course catalog"

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127543 - Nguyễn Trương Ngọc Thảo

Edited by:

Reviewed by:

24127563 - Trần Thị Ngọc Trâm - 25%

- **Task Evidence:**

Task 1: Create Conceptual Model

Description: Identified the main entities and relationships related to academic structure, Academic Record, course offerings, weekly timetable, Study Plan, Advisor Review, and agent traceability. Divided the model into two diagrams to improve readability.

Task 2: Create Class Diagram

Description: Defined the main attributes, operations, relationships, multiplicities, compositions, dependencies, and tool-interface implementations of three class diagrams for the system (Academic Domain, Planning and Timetable Domain, and LLM Agent and Planning Tools).

Task 3: Write core class specifications

Description: Wrote detailed specifications for ten core classes: StudentProfile, AcademicRecord, CourseAttempt, Course, Curriculum, CourseOffering, ClassSection, StudyPlan, PlanReview, and LLMPlanningAgent. Defined their properties, modifiers, constraints, responsibilities, and operations.

Task 4: Upload design artifacts to GitHub

Description: Uploaded and organized the Requirements Analyst design artifacts on GitHub, including PlantUML source files, exported diagrams, conceptual model

- **Visual Evidence:**

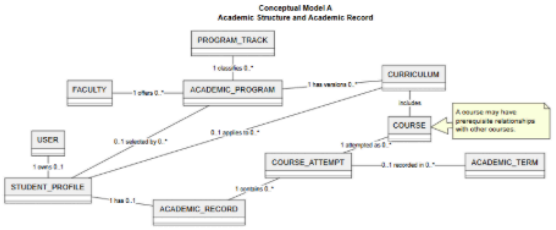


Figure 2.1. Conceptual Model A - Academic Structure and Academic Record

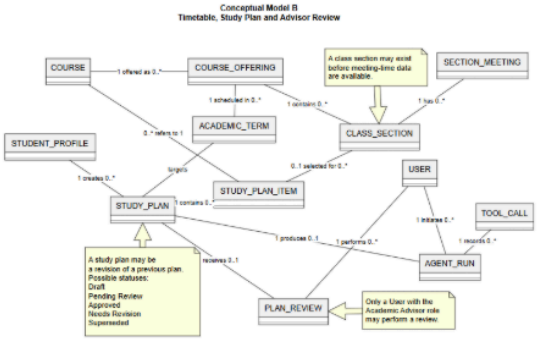


Figure 2.2. Conceptual Model B - Timetable, Study Plan and Advisor Review

1.2 Class Diagram

Written by: 24127563 - Trần Thị Ngọc Trâm

Edited by:

Reviewed by:

The class diagram describes the main object classes of PathToGrad, their responsibilities, and their relationships. To improve readability, the class model is divided into three diagrams. The first diagram presents the academic domain, including academic structure, student profiles, Academic Records, course attempts, courses, curricula, and academic terms. The second diagram presents course offerings, class sections, weekly timetable data, study plans, plan items, advisor reviews, and agent execution records. The third diagram presents the LLM Planning Agent, the tool registry, the common planning-tool interface, deterministic planning tools, fallback planning, logging, advisor review, and plan-version services. The LLM Planning Agent coordinates the planning workflow but does not calculate academic rules directly. Prerequisite checking, graduation progress, timetable conflicts, and academic risks are handled by deterministic tools.

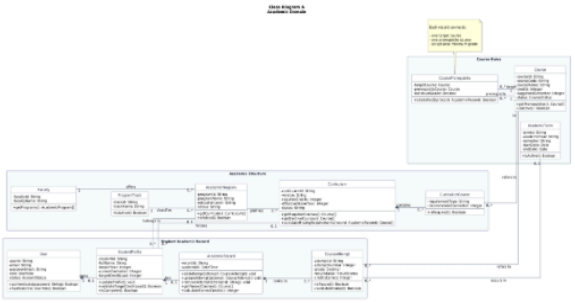


Figure 3.1. Academic Domain Class Diagram

1.3 Class Specifications

Written by: 24127563 - Trần Thị Ngọc Trâm
Edited by: 24127563 - Trần Thị Ngọc Trâm
Reviewed by: 24127427 - Trần Nhựt Đăng Khoa

1.3.1 Class C1

Class name: StudentProfile

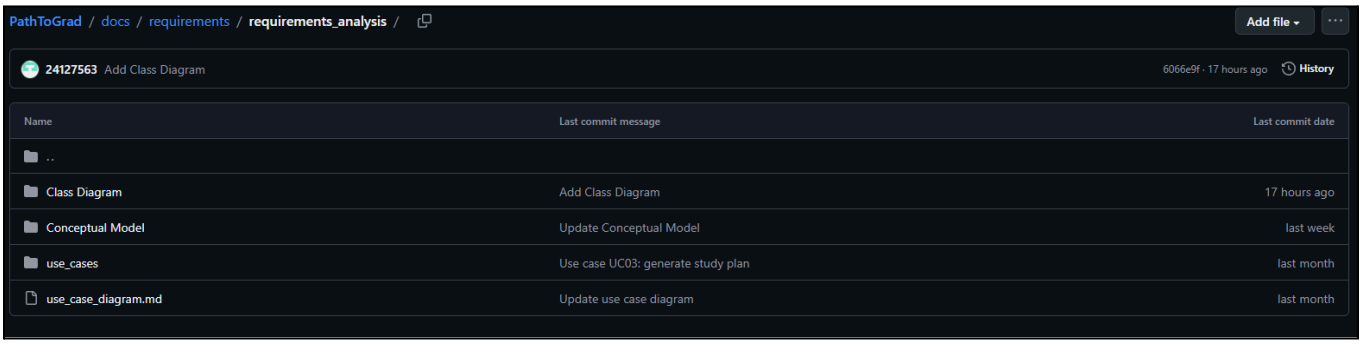
Inheritance: None

Responsibility: Stores the academic context used to generate and validate a student's study plan.

Seq	Property	Modifier	Constraint	Description
1	studentId: String	Private	Required; unique	Identifies the student profile.
2	intakeYear: Integer	Private	Positive year	Stores the student's intake year.
3	currentSemester: Integer	Private	Must be positive	Stores the current study semester.
4	targetCreditLoad: Integer	Private	Must follow configured credit policy	Stores the expected semester credit load.
5	academicProgram: AcademicProgram	Private	May be empty while the profile is incomplete	Stores the selected academic program.
6	curriculum: Curriculum	Private	Must belong to the selected program	Stores the applicable curriculum version.

Seq	Operation	Modifier	Constraint	Description
1	updateProfile()	Public	Input must be valid	Updates academic profile information.
2	validateTargetCreditLoad()	Public	-	Checks whether the selected credit load is valid.

HCMUS | SE Dept. 12



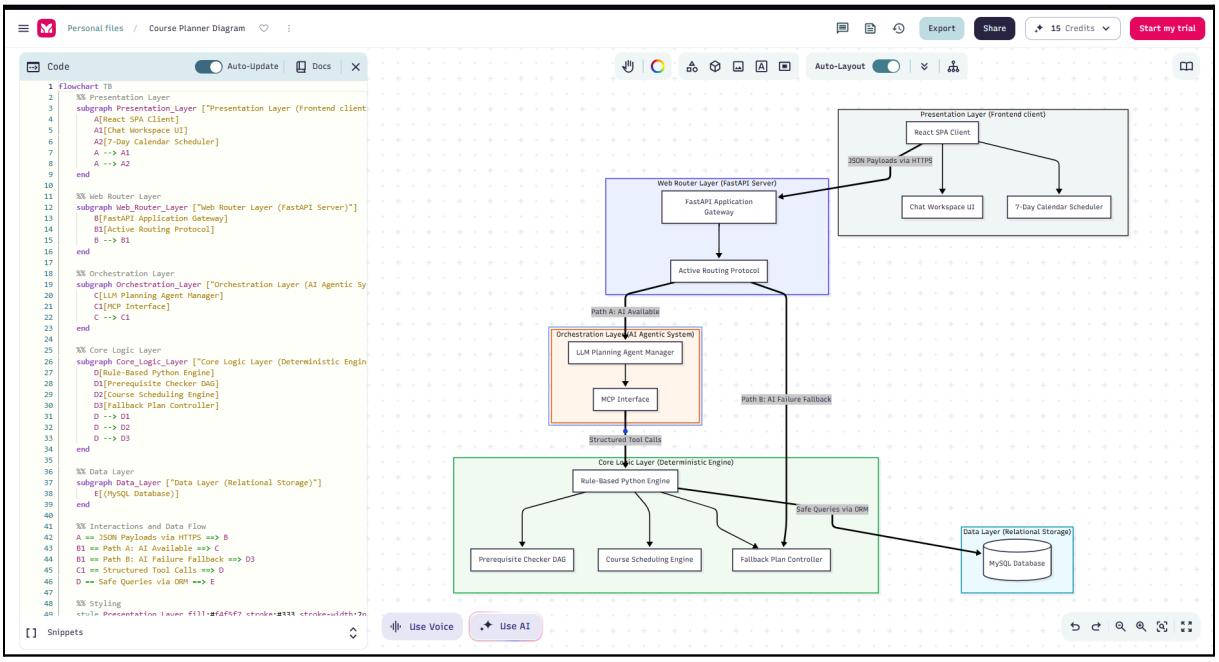
24127080 - Ông Khánh Minh - 25%

- **Task Evidence:**

Task 1: Build an overall architecture diagram of the system

Description: Provides a clear visualization of how the system operates through each system call, illustrating the process logic of the system and parts utilized for the call.

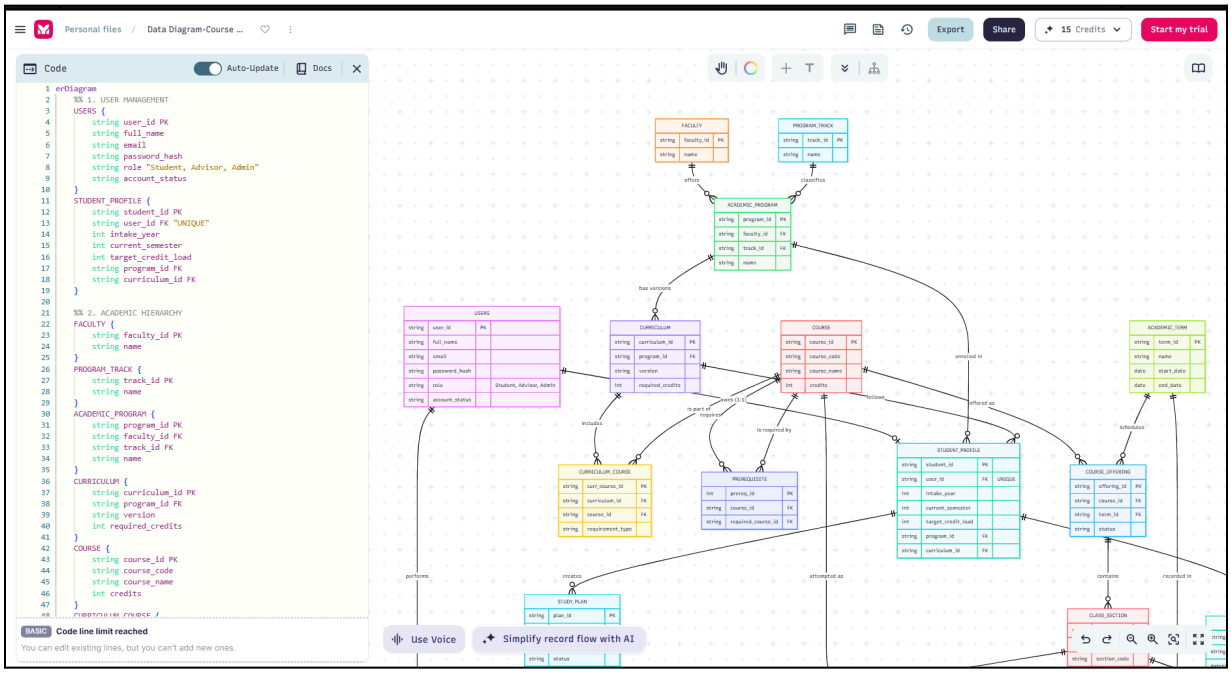
- **Visual Evidence:**



Task 2: Defines Data Structure of the system through diagrams & specifications

Description: Provides a thorough description of the to-be-implemented database for the system operations with detailed specifications of each data type.

- **Visual Evidence:**



2 Conceptual Model

Written by: 24127563 - Trần Thị Ngọc Trâm

Edited by:

Reviewed by:

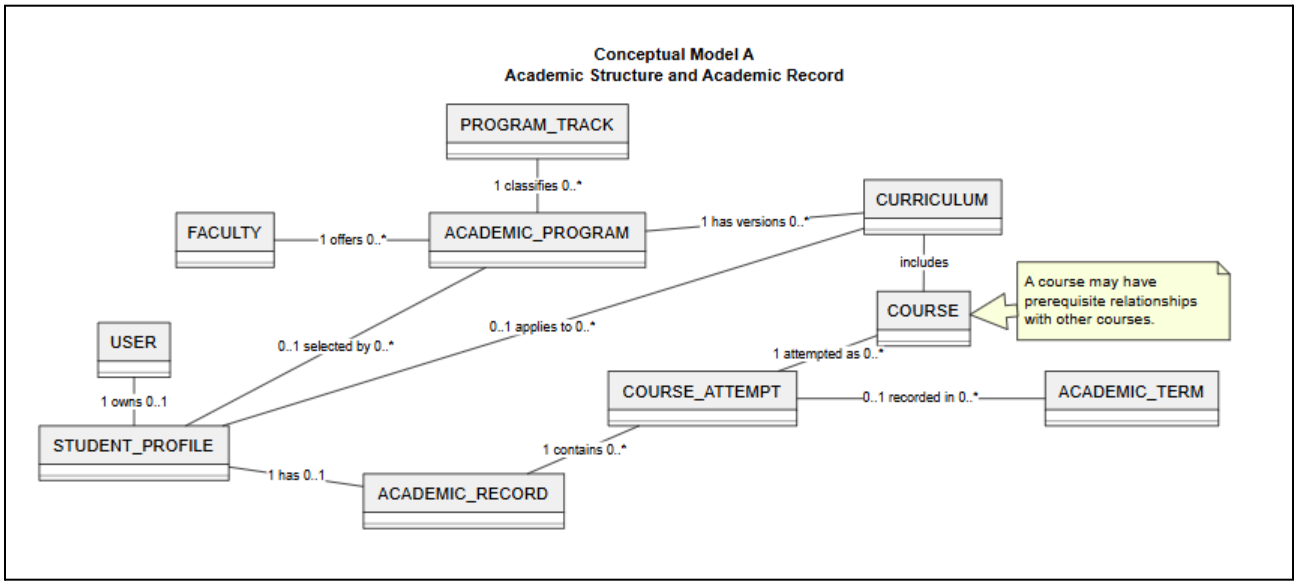
The conceptual model presents the main semantic entities of PathToGrad and their relationships. The model covers user access, academic structure, student Academic Records, curriculum rules, course offerings, weekly timetables, study plans, advisor reviews, and agent traceability.

A Student Profile is linked to an Academic Program and a Curriculum. The student's Academic Record contains Course Attempts used for prerequisite checking and graduation-progress calculation. Course Offerings, Class Sections, and Section Meetings provide the schedule data required to generate a weekly timetable.

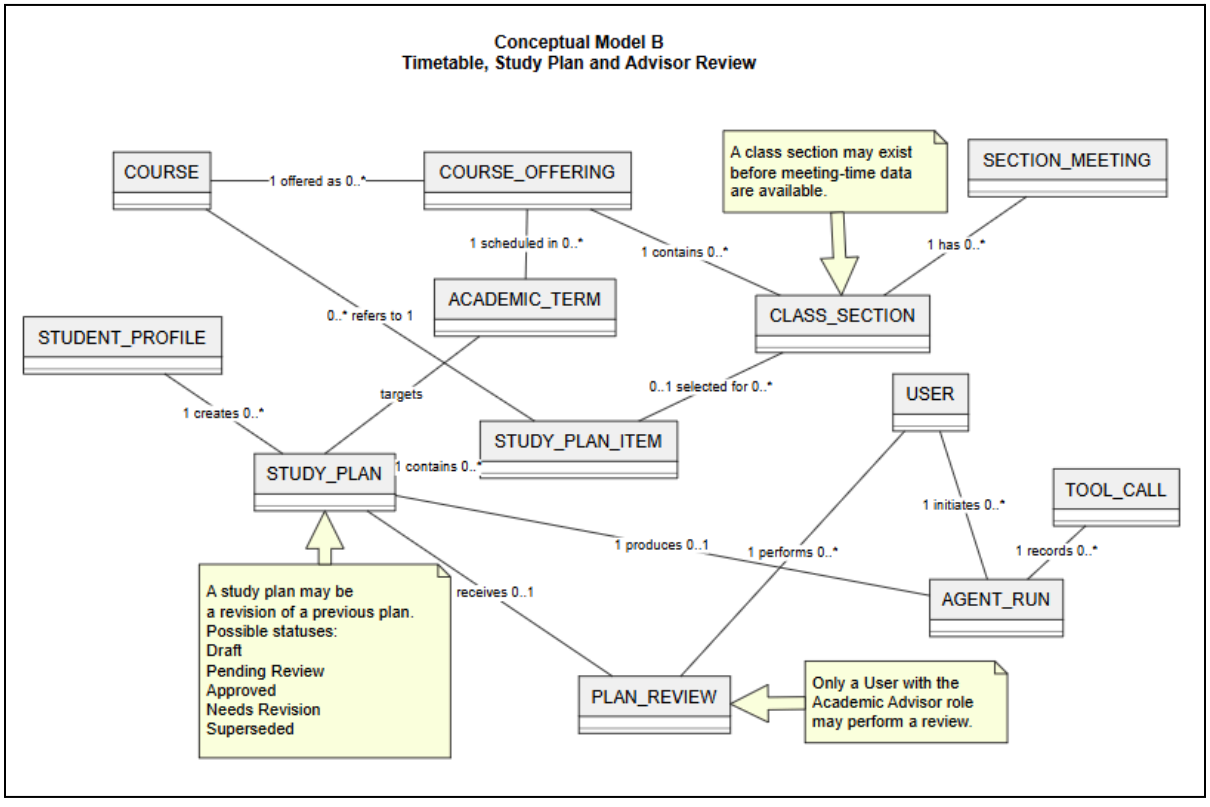
A generated Study Plan is stored as a versioned record. A new plan is saved as Draft and may be submitted for advisor review. A submitted plan becomes Pending Review. The advisor may approve the plan or request revision and provide a comment. When revision is requested, the student creates a new plan version while the earlier plan and review record remain unchanged.

Advisor approval provides academic planning guidance only. It does not perform official course registration. The LLM Planning Agent coordinates internal tools, while prerequisite checks, timetable conflicts, graduation progress, and review records are based on structured system data.

The conceptual model is divided into two diagrams to improve readability. Some shared entities, including User, Student Profile, Course, and Academic Term, may appear in more than one diagram; however, each name represents the same semantic entity in the PathToGrad system.



Conceptual Model A - Academic Structure & Record



Conceptual Model B - Timetable, Study Plan & Advisor Review

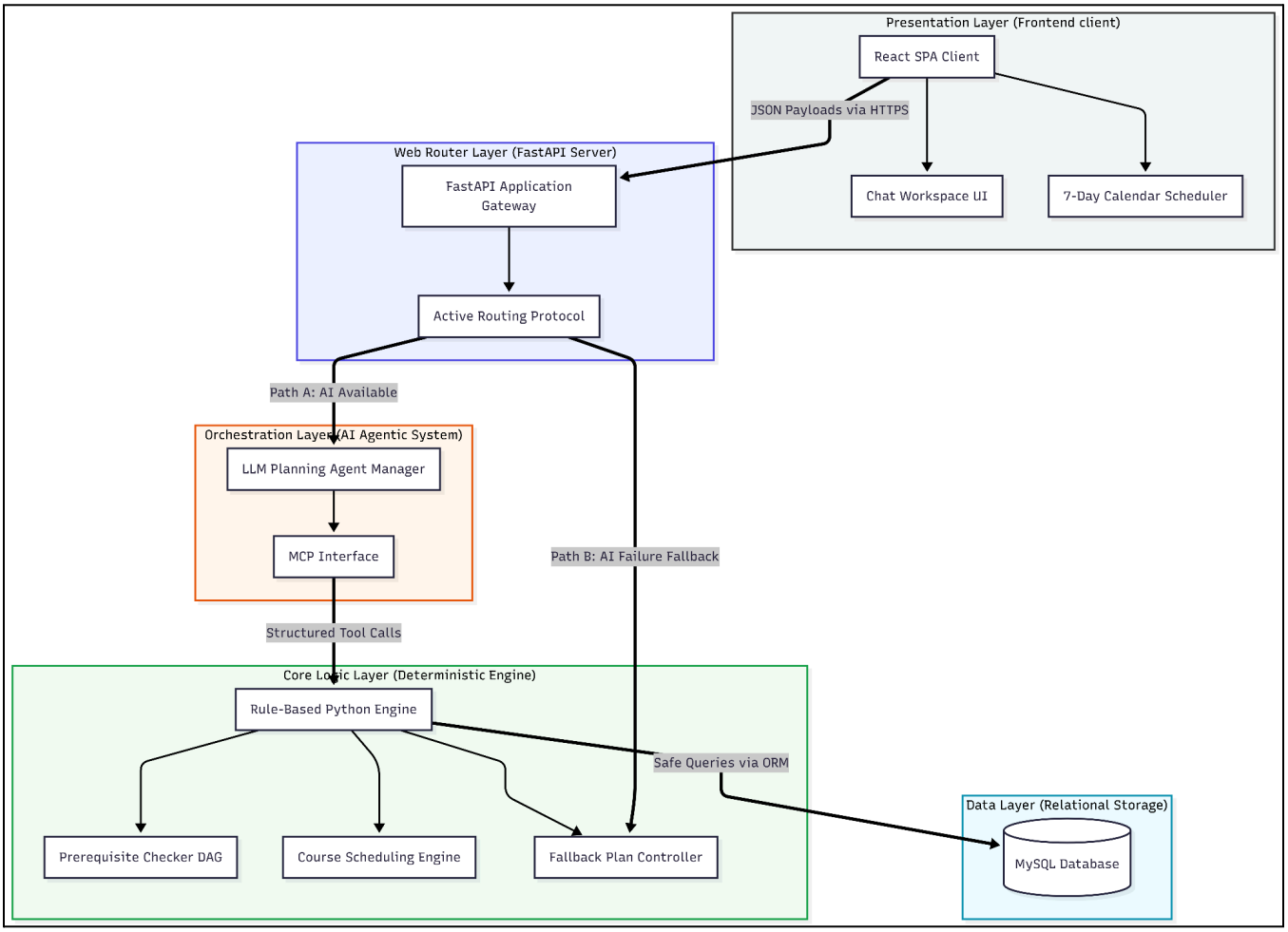
3 Architectural Design

1.1 Architecture Diagram

Written by: 24127080 - Ông Khánh Minh

Edited by:

Reviewed by: 24127427 - Trần Nhựt Đăng Khoa



1.1.1 System Decomposition & Component Specifications

Presentation Layer (Frontend Client)

- **Component:** React SPA (Single Page Application).
- **Responsibility:** Renders the interactive user interface, including the Chat Workspace and the 7-Day Visual Calendar. It collects user inputs, manages local session states, and asynchronously communicates with the backend via RESTful APIs.

Web Router Layer (API Gateway)

- **Component:** FastAPI Server Gateway.
- **Responsibility:** Acts as the central traffic controller. It authenticates incoming requests and monitors the health of the external AI provider.
- **Fault Tolerance:** Implements the **Active Routing Protocol**. If the external LLM API times out or returns an error, this layer automatically reroutes the request to the local Fallback Controller, ensuring zero downtime for the student.

Orchestration Layer (AI Agentic Layer)

- **Component:** LLM Agent Manager & MCP Interface.
- **Responsibility:** Processes natural language inputs from the user. Instead of relying on internal LLM knowledge, it utilizes a Model Context Protocol (MCP) to trigger specific internal tools (e.g., check_prerequisites()). It aggregates the deterministic data returned by these tools and formats it into natural, explainable language for the student.

Core Logic & Data Layer (Deterministic Engine)

- **Component:** Rule-Based Python Engine & MySQL Database.
- **Responsibility:** The absolute source of truth. The deterministic engine calculates graduation progress, validates credit limits (14-24 credits), and traverses prerequisite trees (DAG algorithms).

- **Data Security:** The LLM does not have direct database access. All data retrieval (Course Catalogs, Student Records) must pass through this strictly typed layer to ensure data integrity and prevent hallucinated scheduling before querying the MySQL relational database.

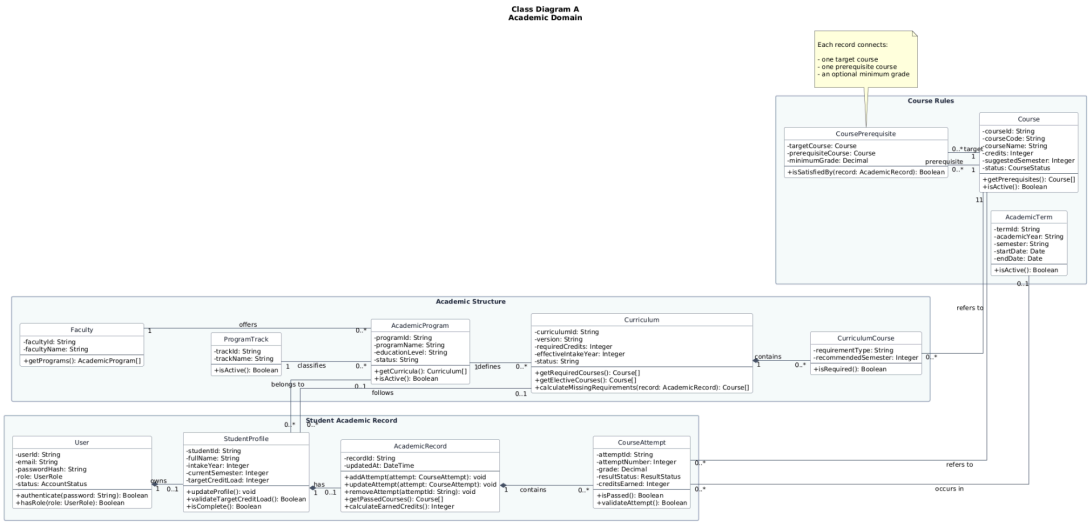
1.2 Class Diagram

Written by: 24127563 - Trần Thị Ngọc Trâm

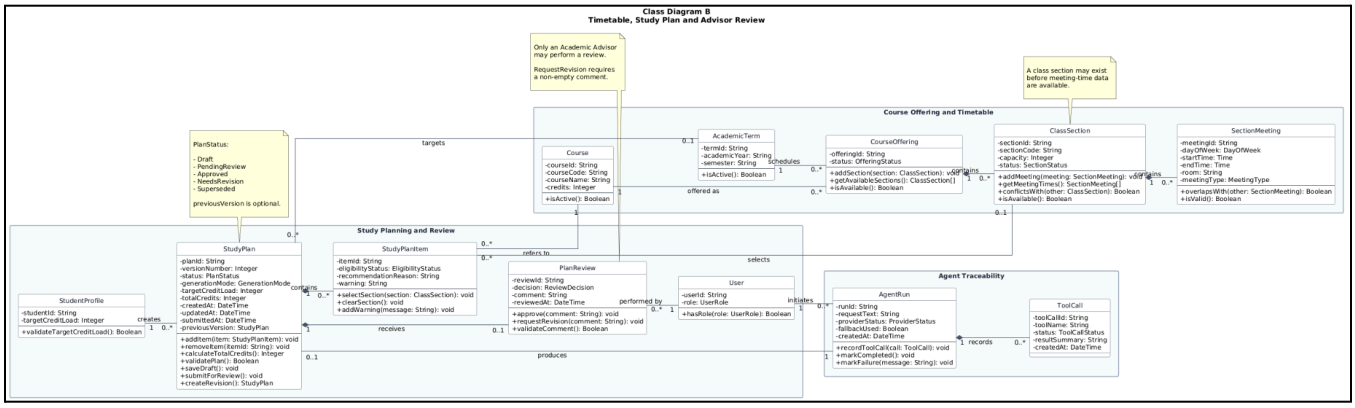
Edited by:

Reviewed by:

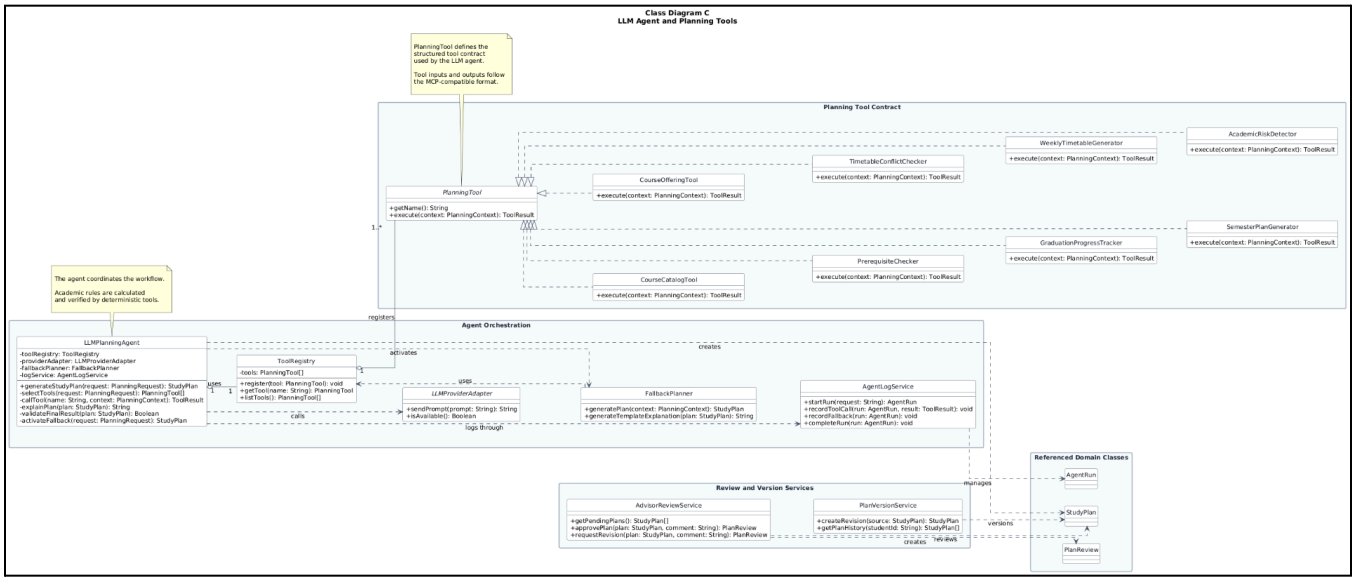
The class diagram describes the main object classes of PathToGrad, their responsibilities, and their relationships. To improve readability, the class model is divided into three diagrams. The first diagram presents the academic domain, including academic structure, student profiles, Academic Records, course attempts, courses, curricula, and academic terms. The second diagram presents course offerings, class sections, weekly timetable data, study plans, plan items, advisor reviews, and agent execution records. The third diagram presents the LLM Planning Agent, the tool registry, the common planning-tool interface, deterministic planning tools, fallback planning, logging, advisor review, and plan-version services. The LLM Planning Agent coordinates the planning workflow but does not calculate academic rules directly. Prerequisite checking, graduation progress, timetable conflicts, and academic risks are handled by deterministic tools.



Academic Domain Class Diagram



Planning & Timetable Domain Class Diagram



LLM Agent & Planning Tool Class Diagram

1.3 Class Specifications

Written by: 24127563 - Trần Thị Ngọc Trâm

Edited by: 24127563 - Trần Thị Ngọc Trâm

Reviewed by: 24127427 - Trần Nhựt Đăng Khoa

1.3.1 Class C1

Class name: StudentProfile

Inheritance: None

Responsibility: Stores the academic context used to generate and validate a student's study plan.

Seq	Property	Modifier	Constraint	Description
1	studentId: String	Private	Required; unique	Identifies the student profile.
2	intakeYear: Integer	Private	Positive year	Stores the student's intake year.
3	currentSemester: Integer	Private	Must be positive	Stores the current study semester.
4	targetCreditLoad: Integer	Private	Must follow configured credit policy	Stores the expected semester credit load.
5	academicProgram: AcademicProgram	Private	May be empty while the profile is incomplete	Stores the selected academic program.
6	curriculum: Curriculum	Private	Must belong to the selected program	Stores the applicable curriculum version.

Seq	Operation	Modifier	Constraint	Description
1	updateProfile()	Public	Input must be valid	Updates academic profile information.
2	validateTargetCreditLoad()	Public	-	Checks whether the selected credit load is valid.

3	getApplicableCurriculum()	Public	Program and intake data should exist	Returns the curriculum applied to the student.
4	isComplete()	Public	-	Determines whether enough information exists for planning.

1.3.2 Class C2

Class name: AcademicRecord

Inheritance: None

Responsibility: Maintains the course attempts used for prerequisite checking and graduation-progress calculation.

Seq	Property	Modifier	Constraint	Description
1	recordId: String	Private	Required; unique	Identifies the record.
2	updatedAt: DateTime	Private	Automatically updated	Records the latest change time.
3	attempts: List<CourseAttempt>	Private	May be empty	Contains all course attempts.

Seq	Operation	Modifier	Constraint	Description
1	addAttempt(attempt)	Public	Attempt must be valid	Adds a course attempt.
2	updateAttempt(attempt)	Public	Attempt must exist	Updates a course attempt.
3	removeAttempt(attemptId)	Public	Must not remove another student's attempt	Removes a course attempt.
4	getPassedCourses()	Public		Returns passed courses.
5	calculateEarnedCredits()		Count verified passed attempts only	Calculates earned credits.

1.3.3 Class C3

Class name: CourseAttempt

Inheritance: None

Responsibility: Represents one attempt by a student to complete a course.

Seq	Property	Modifier	Constraint	Description
1	attemptId: String	Private	Required; unique	Identifies the attempt.
2	attemptNumber: Integer	Private	At least 1	Indicates how many times the course has been attempted.
3	grade: Decimal	Private	Must follow the supported grading scale	Stores the result grade.
4	resultStatus: ResultStatus	Private	Passed, Failed, or InProgress	Stores the attempt status.
5	creditsEarned: Integer	Private	Cannot be negative	Stores credits earned from the attempt.
6	course: Course	Private	Required	Identifies the attempted course.
7	term: AcademicTerm	Private	May be empty for incomplete records	Identifies the academic term.

Seq	Operation	Modifier	Constraint	Description
1	isPassed()	Public	-	Returns whether the attempt passed.
2	validateAttempt()	Public	-	Validates grade, status, term, and credits.
3	calculateEarnedCredits()	Public	Passed attempts only	Returns earned credits.

1.3.4 Class C4

Class name: Course

Inheritance: None

Responsibility: Represents a course and its general academic information.

Seq	Property	Modifier	Constraint	Description
1	courseId: String	Private	Required; unique	Identifies the course.
2	courseCode: String	Private	Required; unique	Stores the official course code.
3	courseName: String	Private	Required	Stores the course name.
4	credits: Integer	Private	Greater than zero	Stores the number of credits.
5	suggestedSemester: Integer	Private	May be empty	Stores the suggested study semester.
6	status: CourseStatus	Private	Active or Archived	Store availability status.
7	prerequisites: List<CoursePrerequisite >	Private	May be empty	Stores prerequisite courses.

Seq	Operation	Modifier	Constraint	Description
1	getPrerequisites()	Public	-	Returns prerequisite courses.
2	isActive()	Public	-	Check whether the course is active.

1.3.5 Class C5

Class name: Curriculum

Inheritance: None

Responsibility: Defines course and credit requirements for an academic-program version.

Seq	Property	Modifier	Constraint	Description
1	curriculumId: String	Private	Required; unique	Identifies the curriculum.
2	version: String	Private	Required	Stores the curriculum version.
3	requiredCredits: Integer	Private	Cannot be negative	Stores total required credits.
4	effectiveIntakeYear: Integer	Private	Required	Identifies the intake from which it applies.
5	curriculumCourses: List<CurriculumCourse>	Private	May initially be empty	Stores the course requirements defined for this curriculum.

Seq	Operation	Modifier	Constraint	Description
1	getRequiredCourses()	Public	-	Returns compulsory courses.
2	getElectiveCourses()	Public	-	Returns elective courses.
3	calculateMissingRequirements(record)	Public	Record must be available	Returns remaining requirements.
4	appliesTo(profile)	Public	-	Checks whether the curriculum applies to a profile.

1.3.6 Class C6

Class name: CourseOffering

Inheritance: None

Responsibility: Represents the availability of one course in an academic term.

Seq	Property	Modifier	Constraint	Description
1	offeringId: String	Private	Required; unique	Identifies the offering.
2	course: Course	Private	Required	Identifies the offered course.

3	term: AcademicTerm	Private	Required	Identifies the academic term.
4	status: OfferingStatus	Private	Active, Inactive, or Archived	Stores offering status.
5	sections: List<ClassSection>	Private	May be empty	Stores available class sections.

Seq	Operation	Modifier	Constraint	Description
1	getAvailableSections()	Public	Active sections only	Returns selectable sections.
2	addSection(section)	Public	Section must be valid	Adds a class section.
3	isAvailable()	Public	-	Checks whether the offering can be used.

1.3.7 Class C7

Class name: ClassSection

Inheritance: None

Responsibility: Represents a selectable class and its meeting schedule.

Seq	Property	Modifier	Constraint	Description
1	sectionId: String	Private	Required; unique	Identifies the class section.
2	sectionCode: String	Private	Required	Stores the section code.
3	courseOffering	Private	Required; must refer to one course offering	Identifies the course offering to which the section belong to.
4	capacity: Integer	Private	Cannot be negative	Stores planned capacity.
5	status: SectionStatus	Private	Active, Inactive, or Archived	Stores section status.
6	meetings: List<SectionMeeting>	Private	May be empty	Stores meeting times.

Seq	Operation	Modifier	Constraint	Description
1	getMeetingTimes(): List<SectionMeeting>	Public	-	Returns section meetings.
2	addMeeting(meeting:SectionMeeting):void	Public	Meeting must be valid	Adds a meeting time to the section
3	conflictsWith(other:ClassSection):Boolean	Public	-	Checks schedule overlap.
4	isAvailable() :Boolean	Public	-	Checks whether the section is selectable.

1.3.8 Class C8

Class name: StudyPlan

Inheritance: None

Responsibility: Stores a versioned semester plan and controls its review lifecycle.

Seq	Property	Modifier	Constraint	Description
1	planId: String	Private	Required; unique	Identifies the plan version.
2	studentProfile	Private	Required	Identifies the student who owns the plan.
3	targetTerm: AcademicTerm	Private	May be empty in Draft	Identifies the academic term targeted by the plan.
4	versionNumber: Integer	Private	At least 1	Stores version number.
5	status: PlanStatus	Private	Draft, PendingReview, Approved, NeedsRevision, or Superseded	Stores lifecycle status.
6	generationMode: GenerationMode	Private	LLM or Fallback	Stores generation mode.

7	targetCreditLoad: Integer	Private	Must follow configured policy	Stores requested credit load.
8	totalCredits: Integer	Private	Derived from plan items	Stores total plan credits.
9	previousVersion: StudyPlan	Private	Optional	Links to the earlier version.
10	items: List<StudyPlanItem>	Private	May be empty in Draft	Stores recommended courses.
11	createdAt	Private	Automatically set when created	Stores the plan creation time.
12	updatedAt: DateTime	Private	Automatically updated	Stores the latest update time.
13	submittedAt: DateTime	Private	submittedAt: DateTime	Stores the time the plan was submitted for review.

Seq	Operation	Modifier	Constraint	Description
1	addItem(item:StudyPlanItem): void	Public	Plan must be editable	Adds a course item.
2	removeItem(itemId:String): void	Public	Plan must be Draft	Removes an item.
3	calculateTotalCredits(): Integer	Public	-	Calculates total credits.
4	validatePlan(): Boolean	Public	Uses verified rules	Validates prerequisites, load, and timetable.
5	saveDraft(): void	Public		Saves the plan as Draft.
6	submitForReview(): void	Public	Blocking errors must be resolved	Changes status to PendingReview.
7	createRevision(): StudyPlan	Public	Source should require revision	Creates a new Draft version.

1.3.9 Class C9

Class name: PlanReview

Inheritance: None

Responsibility: Stores an advisor's decision and comment for one submitted plan version.

Seq	Property	Modifier	Constraint	Description
1	reviewId: String	Private	Required; unique	Identifies the review.
2	decision: ReviewDecision	Private	Approved or NeedsRevision	Stores advisor decision.
3	comment: String	Private	Required for NeedsRevision	Stores advisor feedback.
4	reviewedAt: DateTime	Private	Set when decision is recorded	Stores review time.
5	advisor: User	Private	User must have Advisor role	Identifies the reviewer.
6	studyPlan: StudyPlan	Private	Must be PendingReview	Identifies the reviewed plan.

Seq	Operation	Modifier	Constraint	Description
1	approve(comment)	Public	Plan must be PendingReview	Approves the plan.
2	requestRevision(comment)	Public	Comment is required	Requests plan revision.
3	validateComment()	Public	-	Check comment requirements.

1.3.10 Class C10

Class name: LLMPlanningAgent

Inheritance: None

Responsibility: Coordinates planning requests, selects tools, calls the provider, validates results and activates fallback mode.

Seq	Property	Modifier	Constraint	Description
1	toolRegistry: ToolRegistry	Private	Required	Provides available planning tools.
2	providerAdapter: LLMProviderAdapter	Private	One configured provider	Connects to the LLM provider.
3	fallbackPlanner: FallbackPlanner	Private	Required	Provides rule-based fallback.
4	logService: AgentLogService	Private	Required	Records runs and tool calls.

Seq	Operation	Modifier	Constraint	Description
1	generateStudyPlan(request)	Public	Student data must be available	Coordinates plan generation.
2	selectTools(request)	Private	-	Selects tools required for the request.
3	callTool(name, input)	Private	Tool must be registered	Executes a planning tool.
4	explainPlan(plan)	Private	Explanation must use verified tool results	Produces the final explanation.
5	activateFallback(request)	Private	Used when the provider is unavailable	Generates a rule-based plan.
6	validateFinalResult(plan)	Private	All recommendations must be verified	Prevents unsupported recommendations.

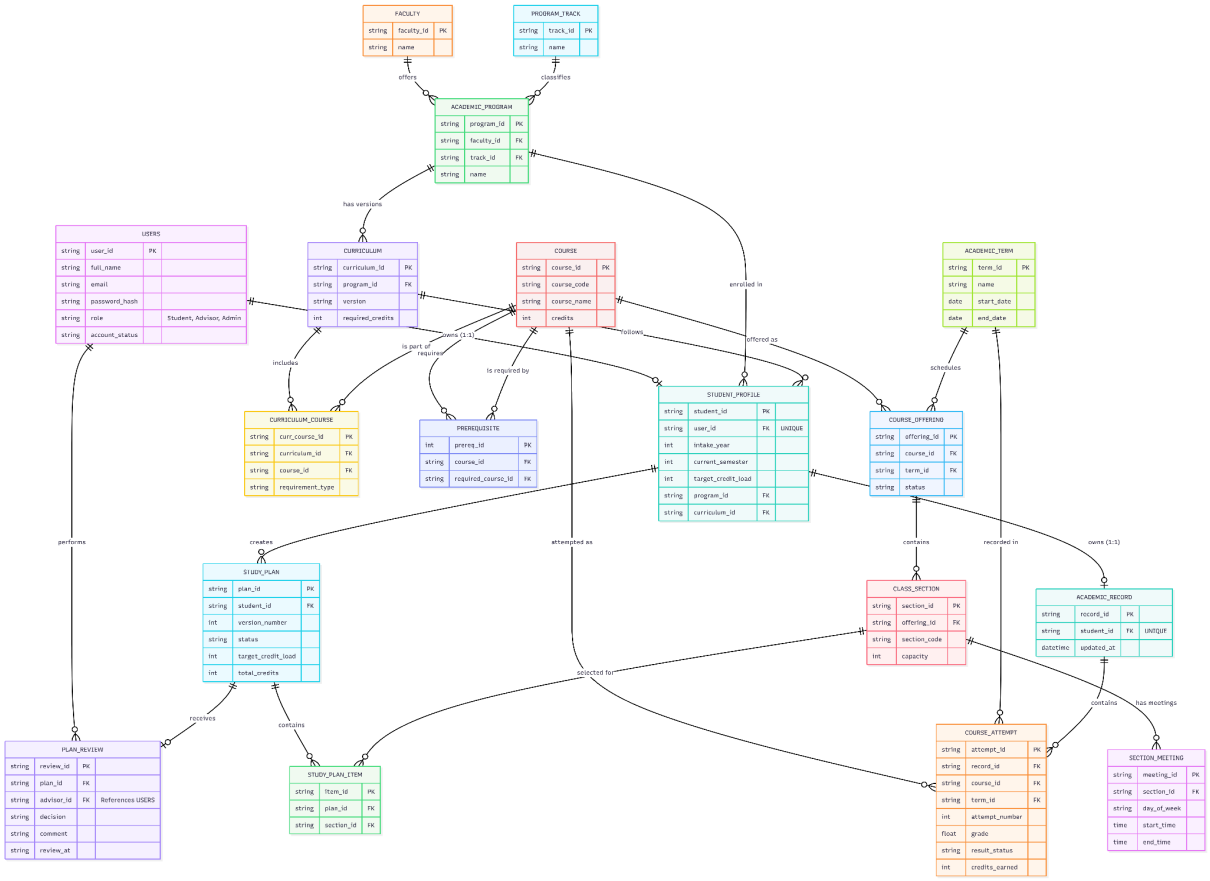
4 Data Design

1.4 Data Diagram

Written by: 24127080 - Ông Khánh Minh

Edited by: 24127080 - Ông Khánh Minh

Reviewed by: 24127427 - Trần Nhựt Đăng Khoa



1.5 Data Specification

Written by: 24127080 - Ông Khánh Minh

Edited by:

Reviewed by: 24127427 - Trần Nhựt Đăng Khoa

User Management & Profiles

1. USERS

Attribute Name	Data Type	Constraints	Description
user_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier (UUID) for the user.
full_name	NVARCHAR(50)	NOT NULL	Full name of the user.
email	NVARCHAR(50)	NOT NULL	Contact email of the user.
password	VARCHAR(255)	NOT NULL	Securely hashed representation of the user's password.
role	ENUM	NOT NULL	Restricted to 'Student', 'Advisor', or 'Admin'.
account_status	ENUM	NOT NULL	The current login permission state of the account. Restricted to 'Active' or 'Suspended'.

2. STUDENT_PROFILE

Attribute Name	Data Type	Constraints	Description
student_id	VARCHAR(20)	PRIMARY KEY, NOT NULL	The official university student ID string.
user_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the USER table.

intake_year	INT	NOT NULL, > 1900	The year the student enrolled.
current_semester	INT	NOT NULL, > 0	The current study semester number.
target_credit_load	INT	NOT NULL, >= 14, <= 24	Expected semester credit load.
program_id	VARCHAR(36)	NULL	Links to a specific academic program (optional if incomplete).

Academic Hierarchy & Curriculum

3. FACULTY

Attribute Name	Data Type	Constraints	Description
faculty_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the university faculty/school.
name	VARCHAR(255)	NOT NULL	Name of the faculty (e.g., Faculty of Information Technology).

4. PROGRAM_TRACK

Attribute Name	Data Type	Constraints	Description
track_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the track.
name	VARCHAR(255)	NOT NULL	Name of the track (e.g., Standard, High-Quality, Honors).

5. ACADEMIC_PROGRAM

Attribute Name	Data Type	Constraints	Description
program_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the specific major/program.
faculty_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the FACULTY table.
track_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the PROGRAM_TRACK table.
name	VARCHAR(255)	NOT NULL	Name of the program (e.g., Software Engineering).

6. CURRICULUM

Attribute Name	Data Type	Constraints	Description
curriculum_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the curriculum rule set.
program_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the ACADEMIC_PROGRAM.
version	VARCHAR(50)	NOT NULL	The specific catalog year (e.g., "2024-2025").
required_credits	INT	NOT NULL, > 0	Total credits required to graduate under this version.

7. CURRICULUM_COURSE

Attribute Name	Data Type	Constraints	Description
curr_course_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier mapping a course to a curriculum.
curriculum_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the CURRICULUM.

course_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the COURSE table.
requirement_type	ENUM	NOT NULL	Restricted to 'Core' (compulsory) or 'Elective'.

8. COURSE

Attribute Name	Data Type	Constraints	Description
course_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Internal unique system identifier.
course_code	VARCHAR(15)	UNIQUE, NOT NULL	Official university code (e.g., CSC10001).
course_name	VARCHAR(255)	NOT NULL	The full descriptive name of the course.
credits	INT	NOT NULL, > 0	The credit weight of the course.
status	ENUM	NOT NULL	Restricted to 'Active' or 'Archived'.

9. PREREQUISITE

Attribute Name	Data Type	Constraints	Description
prereq_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the prerequisite rule.
course_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	The target course a student wants to take.
required_course_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	The course that must be passed first.

Temporal & Scheduling

10. ACADEMIC_TERM

Attribute Name	Data Type	Constraints	Description
term_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the semester.
name	VARCHAR(100)	NOT NULL	Human-readable name (e.g., "Fall 2026").
start_date	DATE	NOT NULL	The official first day of classes.
end_date	DATE	NOT NULL	The official last day of final exams.

11. COURSE_OFFERING

Attribute Name	Data Type	Constraints	Description
offering_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for this specific offering.
course_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the base COURSE.
term_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the ACADEMIC_TERM.
status	ENUM	NOT NULL	Restricted to 'Active', 'Canceled', or 'Archived'.

12. CLASS_SECTION

Attribute Name	Data Type	Constraints	Description
section_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the specific section.
offering_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the specific COURSE_OFFERING for that term.

section_code	VARCHAR(20)	NOT NULL	The class grouping code (e.g., "Group 1").
capacity	INT	NOT NULL, >= 0	Maximum number of allowable students.

13. SECTION_MEETING

Attribute Name	Data Type	Constraints	Description
meeting_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the time block.
section_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the CLASS_SECTION (e.g., Group 1).
day_of_week	ENUM	NOT NULL	'Monday', 'Tuesday', 'Wednesday', etc.
start_time	TIME	NOT NULL	The exact minute the class begins.
end_time	TIME	NOT NULL	The exact minute the class ends.

Academic Records

14. ACADEMIC_RECORD

Attribute Name	Data Type	Constraints	Description
record_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the transcript container.
student_id	VARCHAR(20)	FOREIGN KEY, UNIQUE, NOT NULL	Links to STUDENT_PROFILE. Enforces a strict 1:1 relationship.
updated_at	DATETIME	NOT NULL	Timestamp of the last time a grade or attempt was added.

15. COURSE_ATTEMPT

Attribute Name	Data Type	Constraints	Description
attempt_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for this specific attempt.
record_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the student's ACADEMIC_RECORD.
course_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the COURSE taken.
term_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the ACADEMIC_TERM (e.g., Spring 2026).
attempt_number	INT	NOT NULL, >= 1	Tracks if this is a first try or a retake.
grade	DECIMAL(3,1)	NULL	Result grade (NULL if the term is currently in progress).
result_status	ENUM	NOT NULL	Restricted to 'Passed', 'Failed', or 'InProgress'.
credits_earned	INT	NOT NULL, >= 0	Credits gained (0 if failed/in progress).

Study Planning & Review

16. STUDY_PLAN

Attribute Name	Data Type	Constraints	Description
plan_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the plan version.
student_id	VARCHAR(20)	FOREIGN KEY, NOT NULL	Links to the requesting student.

version_number	INT	NOT NULL, >= 1	Tracks the revision iteration of the plan.
status	ENUM	NOT NULL	'Draft', 'PendingReview', 'Approved', 'NeedsRevision'.
target_credit_load	INT	NOT NULL	The initial credit limit requested by the student.
total_credits	INT	NOT NULL	The total credits accumulated throughout the studying process

17. STUDY_PLAN_ITEM

Attribute Name	Data Type	Constraints	Description
item_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the plan item.
plan_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to STUDY_PLAN.
section_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the scheduled CLASS_SECTION.

18. PLAN_REVIEW

Attribute Name	Data Type	Constraints	Description
review_id	VARCHAR(36)	PRIMARY KEY, NOT NULL	Unique identifier for the review event.
plan_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the submitted STUDY_PLAN.
advisor_id	VARCHAR(36)	FOREIGN KEY, NOT NULL	Links to the USER table (Advisor role).
decision	ENUM	NOT NULL	Restricted to 'Approved' or 'NeedsRevision'.

comment	TEXT	NULL	Advisor's written feedback. (Enforced as required in the logic layer if the decision is 'NeedsRevision').
review_at	DATETIME	NOT NULL	The exact server timestamp of when the review was submitted.

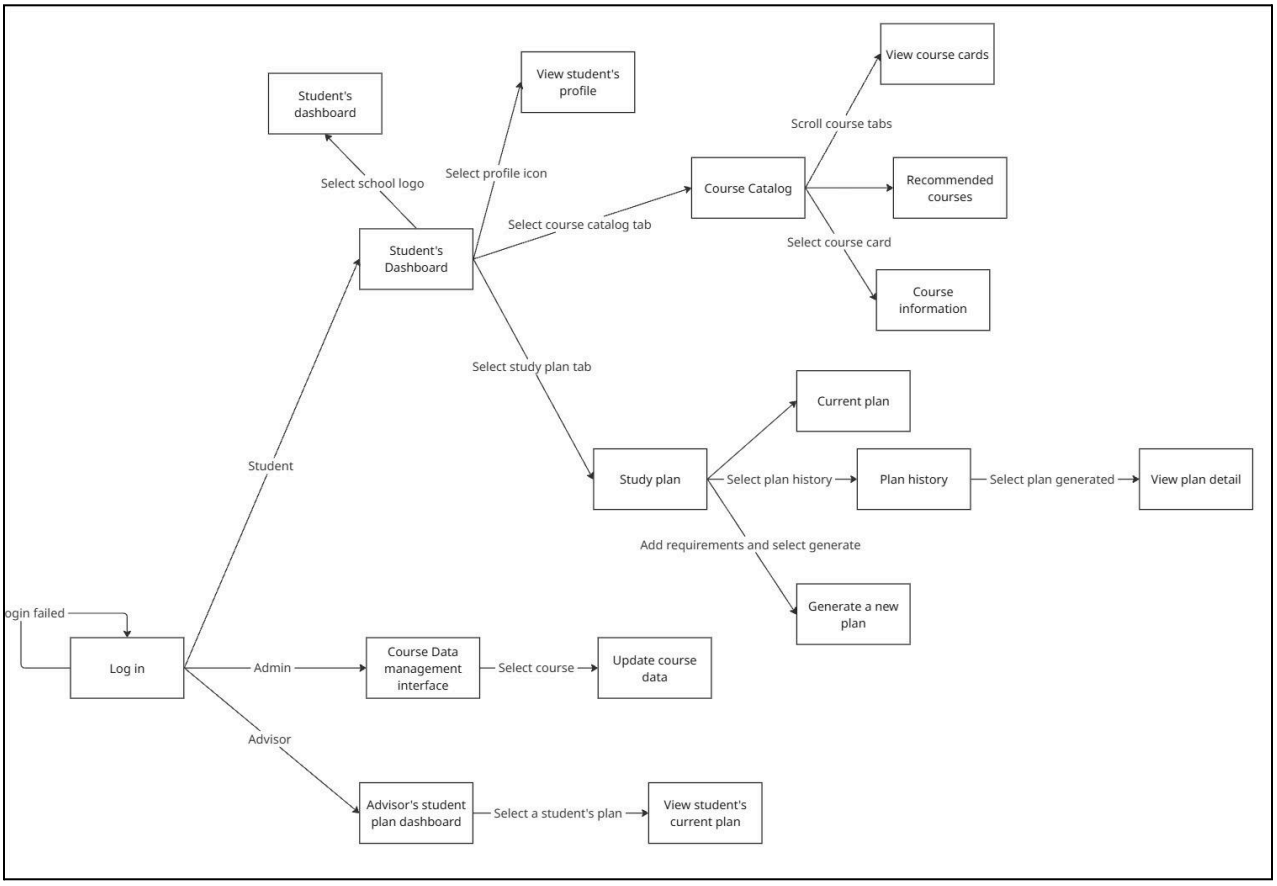
5 User Interface and User Experience Design

1.6 Screen Diagram

Written by: 24127543 - Nguyễn Trương Ngọc Thảo

Edited by:

Reviewed by: 24127427 - Trần Nhựt Đăng Khoa



Seq	Screen	Description
1	**Log in**	Log in interface. Since we assume that once the student has been registered in the school's database, we will automatically create the account so we don't need to sign up for this project. There is also a role choosing button to specify where the next flow will go. It is student by default.
2	**Student's dashboard**	Main interface. Every logo click after this screen will navigate to the student's dashboard. Students can see current total credits, GPA and academic risks. There is also a graduation progress and current study plan's progress with upcoming class, next tasks/submissions and their due dates, along with a work progress reset each week. Advisors's note will also be a pop up window right on the screen for easy reach. The AI chatbox will take up half the screen, with this split view, the student can easily request the AI to check any information without complicated actions (manually checking each part)
3	**Course Catalog**	This is where students register courses. Each card of the scroll bar will show the student right below the information about the class, like the current number of students, advisor, class dates,... The student can also filter the courses for easier view and register, with recommended courses as long as the available courses.
4	**Study plan**	This is where the students make their plans. THey can make plan for the next semester based on their already registered courses. The built in AI will help generate the course, but it also has to handle fall-back and run on algorithms, differ from the AI chatbox next to it.
5	**Plan history**	All discarded plans will be stored here. They can revisit the plan to check information, and also look at the advisor's note on making a study plan for this semester.
6	**Profile page**	This is the student's profile page. It will show all personal information and their guardians.
7	**Student plan's dashboard**	This is where the advisors visit when logging in. Every student in their advising classes will be submitted and checked manually after being checked by AI. The red button means there are many risks and need further attention, while green is almost done and blue is checked/approved plans.

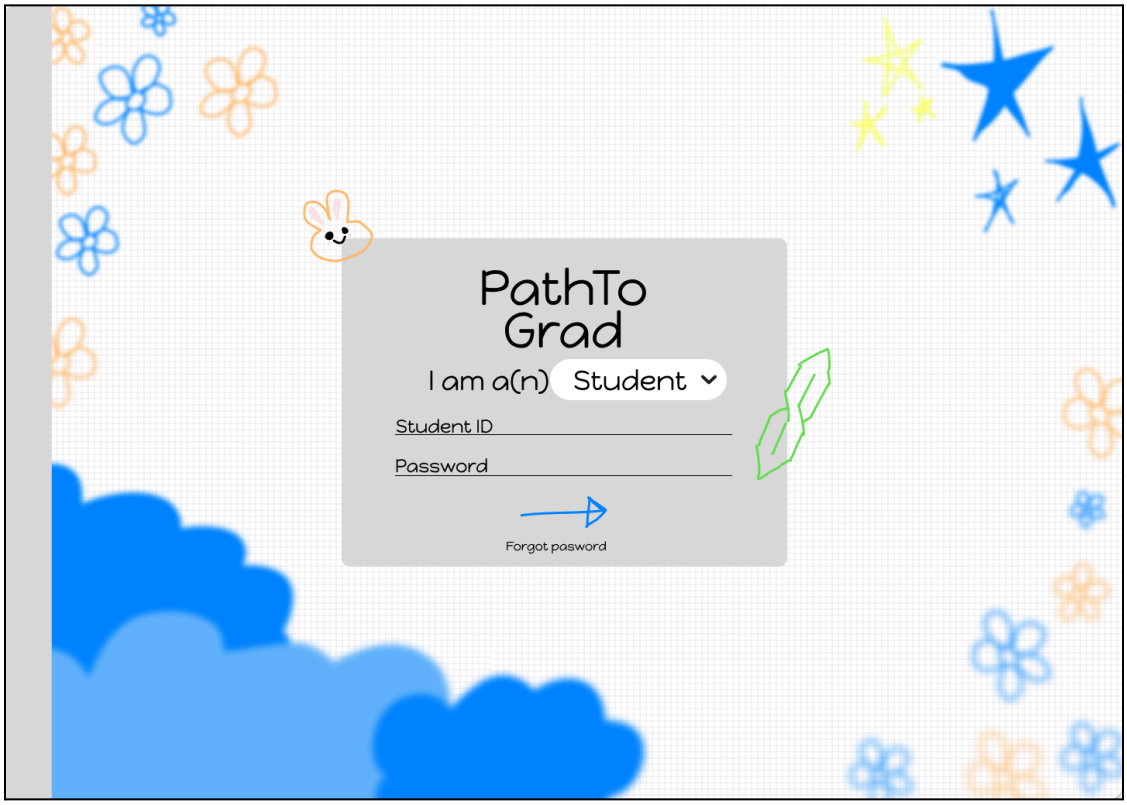
1.7 Screen Specifications

Written by: 24127543 - Nguyễn Trương Ngọc Thảo

Edited by:

Reviewed by: 24127427 - Trần Nhựt Đăng Khoa

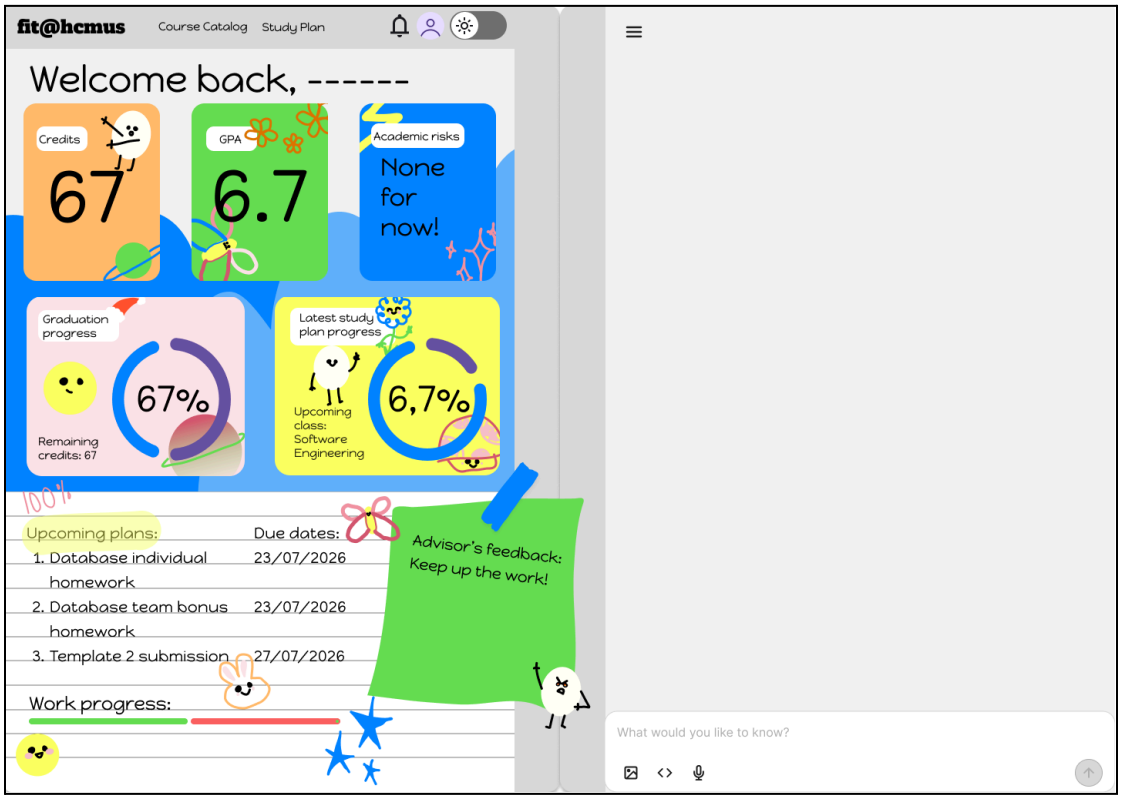
1.7.1 Screen "Login"



The idea of designing this whole project's interface is to make it child-like, DIY notebook ideas. So I chose to make the login interface as the notebook cover, with a nametag on top. This nametag will serve as the login fields. The "I am a(n) _" part, when you click on the small point down icon next to the box, it will open a selection box for role choosing, including student, advisor and admin. The ID will change according to that. After filling up the login information, clicking the blue arrow will transition to the main course of each role.

For interactable demos, please access this link

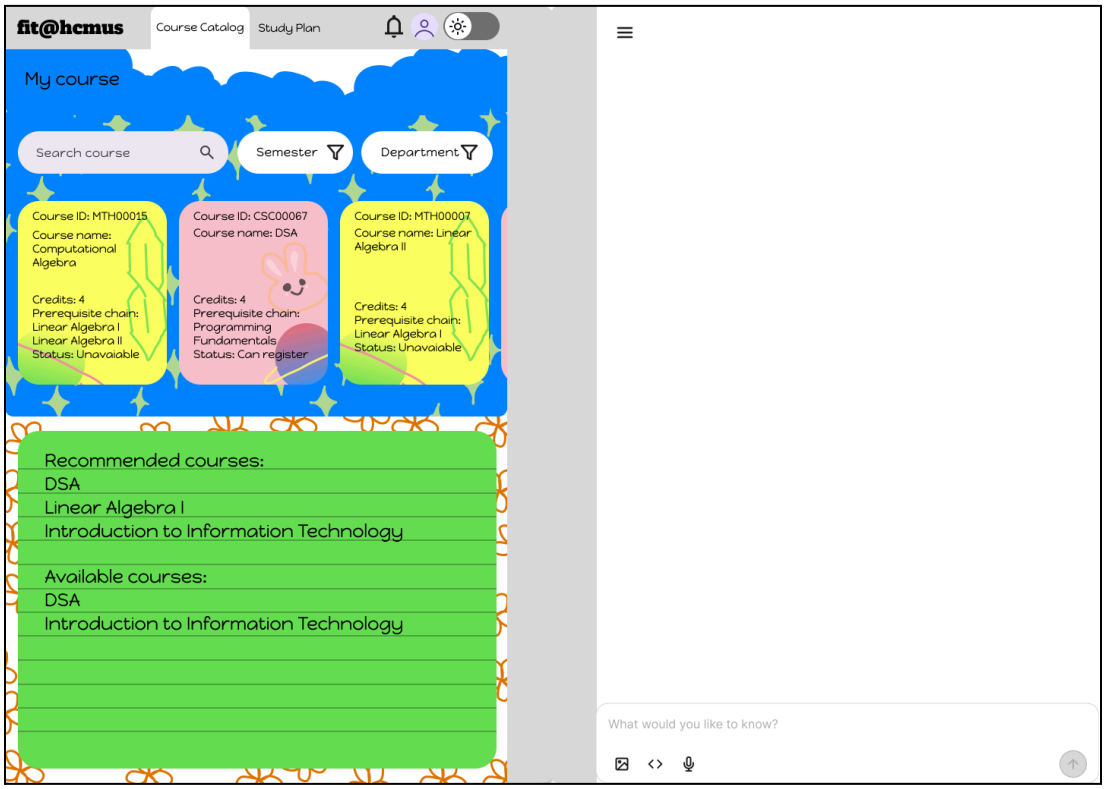
1.7.2 Screen "Student's dashboard"



Following the current vibe and flow, the dashboard is what the students look at the most. They need as much information and a ready-to-go AI chatbox to ask and answer questions right at the beginning. All the colorful tabs are non-interactable, this is just an information board for students to look at. As for the header, the school logo will return every page to this dashboard, Course Catalog will transition to the next Course catalog screen, so will the Study plan. The Profile icon will navigate to the student's profile page, while the bell icon will show the notifications and the changing mode to change the current interface to dark mode.

For interactable demos, please access this link

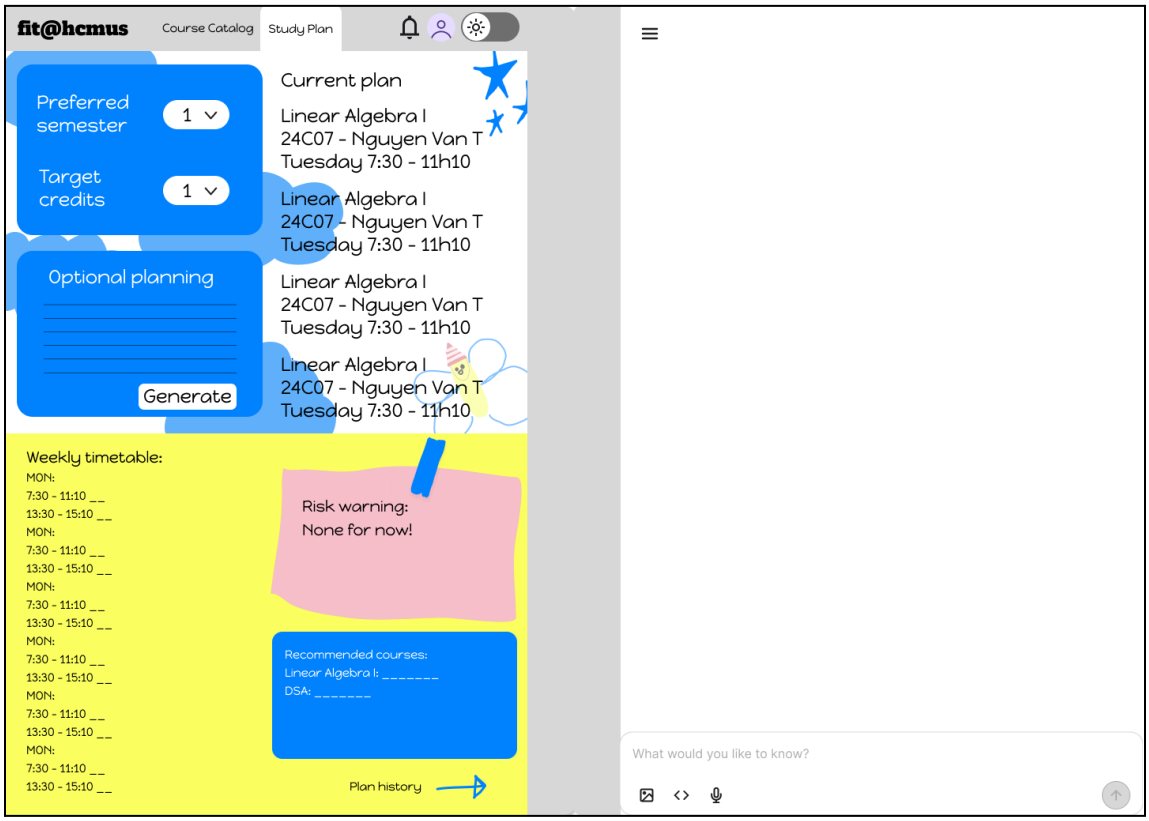
1.7.3 Screen "Course catalog"



This is a more simple and straightforward interface. The search and filter fields will serve its original purpose, while cards show brief information about the course such as name, code, credits students will acquire, courses needed to be studied before studying this course and the current status. By default, the board below will show recommended courses and available courses for easy search and decisions, but by clicking each of the course cards will pop up detailed course information such as teacher, current number of students registered, in class dates,...

For interactable demos, please access this link

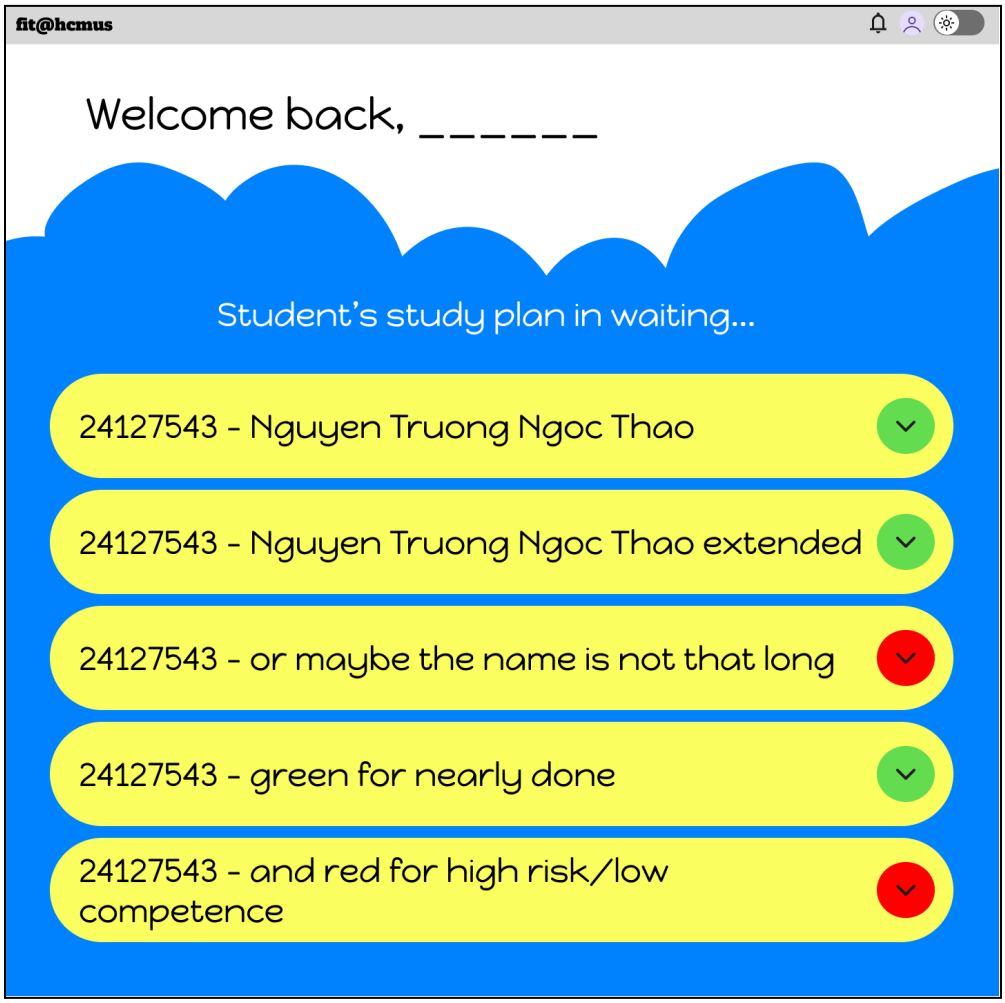
1.7.4 Screen "Study plan"



After registering courses, students can navigate to make plans based on the current courses. This part supports both AI generated alongside the fall-back algorithm that will help students make a proper plan without any overlapped classes. Advisors will also look at this plan once submitted. The weekly timetable will show the accepted plan revised by the advisors themselves, with notes if they have any, and the plan history button will navigate to the plan history interface.

For interactable demos, please access this link

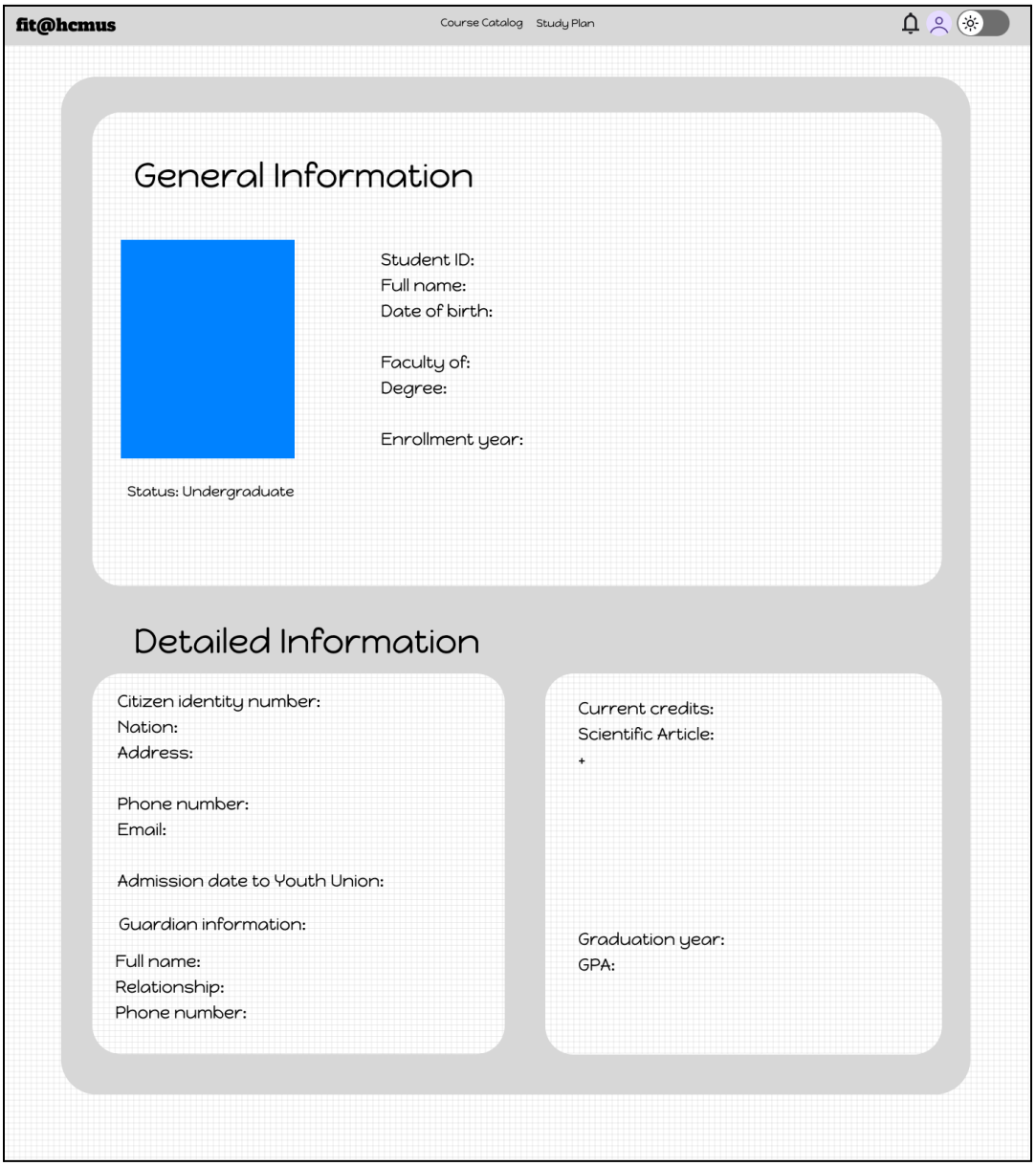
1.7.5 Screen "Student's plan dashboard"



This is the advisors' interface. It is more simple, straightforward and easy to understand to every advisor. The red button means there are many risks and need further attention, while green is almost done and blue is checked/approved plans. After clicking on those tabs, it will navigate to the current plan of the student for approval. The red button means there are many risks and need further attention, while green is almost done and blue is checked/approved plans.

For interactable demos, please access this link

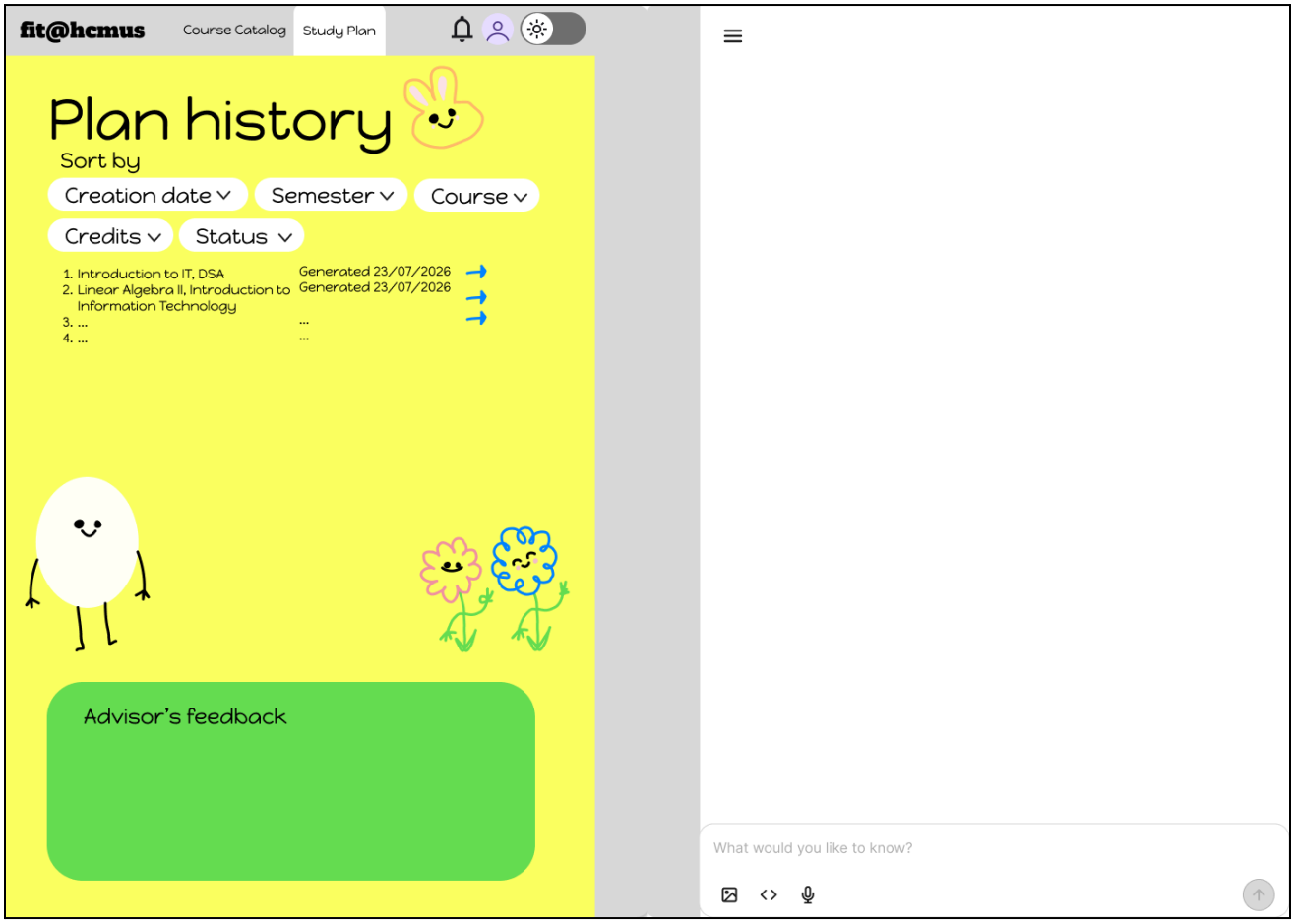
1.7.6 Screen "Profile student"



This part requires a more clear and formal structure. The profile page of the student records their information, including personal and achievements.

For interactable demos, please access this link

1.7.7 Screen "Plan history"



6 AI Usage Declaration

Based on the provided AI Usage Guideline document, teams must submit their AI usage declaration here. Students are encouraged to leverage AI tools creatively and effectively in this project.

7 Presentation

Teams are required to record presentation videos of their work using the provided template.
Every team member must participate in the presentation, specifically covering the sections they contributed to the project.
Videos must not exceed 30 minutes.
Please upload your videos to Youtube (set to either Unlisted or Public).
Provide the Youtube links to the field below.

8 Reflective Report

Written by: 24127427 - Trần Nhựt Đăng Khoa

Edited by:

Reviewed by:

The Reflective Report specifically asks which template sections are most useful for implementation and which are unnecessary, supported by examples.

The sections that are most helpful for implementation are the **Architectural Design**, **Class Diagram**, **Class Specifications**, **Data Design**, **User Interface** and **User Experience Design**. Together, these sections translate the system requirements into structures that developers can directly follow during implementation.

The Architectural Design is useful because it defines the major system components and the communication boundaries between them. For example, the PathToGrad architecture separates the React single-page application, FastAPI server gateway, LLM Planning Agent, deterministic Python engine, fallback controller, and MySQL database. This gives the implementation team a clear understanding of where authentication, request routing, AI coordination, academic-rule validation, fallback planning, and persistent storage should be implemented. It also establishes that the LLM must not access the database directly and that academic decisions must be verified through deterministic tools.

The Class Diagram and **Class Specification**s are particularly useful for backend implementation. They identify the responsibilities, properties, operations, and

relationships of important classes such as StudentProfile, AcademicRecord, CourseAttempt, CourseOffering, ClassSection, StudyPlan, PlanReview, and LLMPlanningAgent. For example, the PlanReview specification states that an advisor may approve a Pending Review plan or request revision with a required comment. This can be directly converted into service methods, validation rules, and application logic.

The Data Diagram and **Data Specifications** are the most directly applicable sections when constructing the MySQL database. They define the tables, primary keys, foreign keys, data types, and constraints required by the system. For example, Course Offering connects a Course to an Academic Term, while Class Section and Section Meeting store the scheduling information required for timetable generation and conflict detection. AcademicRecord and CourseAttempt preserve a student's academic history, while StudyPlan, StudyPlanItem, and PlanReview support planning and advisor-review workflows.

The Screen Diagram and **Screen Specification**s are useful for frontend development because they define the navigation flow, displayed information, and user actions for each role. The Study Plan screen shows the information required to generate and review a plan, while the advisor interface identifies the actions required to approve a plan or request revision. These descriptions can be used to define frontend routes, components, buttons, validation messages, and API requests.

The Conceptual Model is helpful during the early design stage because it explains the meaning of the main entities without implementation details. However, part of its information is repeated in the **Class Diagram** and **Data Diagram**. Once the detailed models are completed, the Conceptual Model becomes more useful as a consistency reference than as a direct implementation guide.

The Architectural, Class, Data, and **UI/UX** sections provide different implementation perspectives. However, repeated descriptions between the Conceptual Model, Class Diagram, Data Diagram, and screen explanations could be reduced to make the document easier to maintain and less likely to contain inconsistencies.